UNIT II

REQUIREMENTS ANALYSIS AND SPECIFICATION

Requirement analysis and specification – Requirements gathering and analysis – Software Requirement Specification – Formal system specification – Finite State Machines – Petrinets – Object modelling using UML – Use case Model – Class diagrams – Interaction diagrams – Activity diagrams – State chart diagrams – Functional modelling – Data Flow Diagram- CASE TOOLS.

2.1 Software requirements

Requirements engineering is the process of gathering, analyzing, documenting, validating, and managing requirements. Requirements analysis is a software engineering task that bridges the gap between system level requirements engineering and software design.

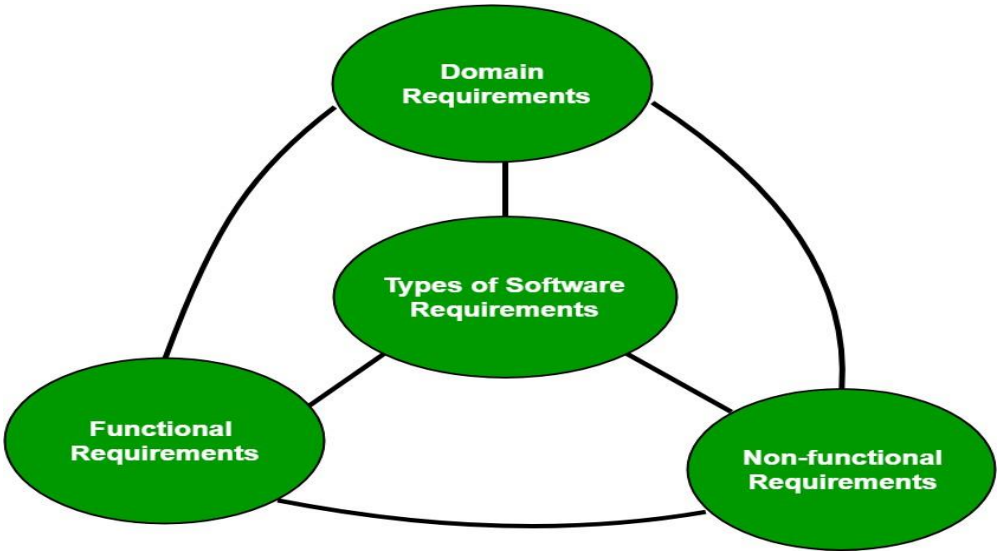
The software requirements are description of features and functionalities of the target system. Requirements convey the expectations of users from the software product. The requirements can be obvious or hidden, known or unknown, expected or unexpected from client's point of view.

Main types of software requirement can be of 3 types

Functional requirements

Non-functional requirements

Domain requirements



Common classifications of software requirements can be:

1. **User requirements:** These requirements describe what the end-user wants from the software system. User requirements are usually expressed in natural language and are typically gathered through interviews, surveys, or user feedback.
2. **System requirements:** These requirements specify the technical characteristics of the software system, such as its architecture, hardware requirements, software components, and interfaces. System requirements are typically expressed in technical terms and are often used as a basis for system design.
3. **Business requirements**: These requirements describe the business goals and objectives that the software system is expected to achieve. Business requirements are usually expressed in terms of revenue, market share, customer satisfaction, or other business metrics.
4. **Regulatory requirements:** These requirements specify the legal or regulatory standards that the software system must meet. Regulatory requirements may include data privacy, security, accessibility, or other legal compliance requirements.
5. **Interface requirements**: These requirements specify the interactions between the software system and external systems or components, such as databases, web services, or other software applications.
6. **Design requirements:** These requirements describe the technical design of the software system. They include information about the software architecture, data structures, algorithms, and other technical aspects of the software.

2.1.1 Functional Requirements

It defines describes functionality or system services. Functional requirements are the behavior or functions that the system must support. These are the attributes that characterize what the software does to fulfill the needs of the customer. These can be business rules, administrative tasks, transactions, cancellations, authentication, authorization, external interfaces, legal or regulatory requirements, audit tracking, certification, reporting requirements, and historical data.

There are many ways of expressing functional requirements e.g., natural language, a structured or formatted language with no rigorous syntax and formal specification language with proper syntax. These are represented or stated in the form of input to be given to the system, the operation

performed and the output expected. They are basically the requirements stated by the user which one can see directly in the final product, unlike the non-functional requirements.

2.1.2 Nonfunctional Requirements

Nonfunctional requirements specify how a system must behave. These are qualities, standards, constraints upon the systems services that are specified with respect to a product, organization, and external environment. Nonfunctional requirements are related to functional requirements, i.e., how efficiently, by how much volume, how fast, at what quality, how safely, etc., a function is performed by a particular system. The examples of nonfunctional requirements are reliability, maintainability, performance, usability, security, scalability, capacity, availability, recoverability, serviceability, manageability, integrity, and interoperability.

Nonfunctional Requirements for ETransQ

- Availability: ETransQ is online information, which will be accessible 24 x 7 hours.
- Recoverability: The system will maintain backup of dynamically changing requirement and any loss of data.
- Extensibility: The system is able to adopt new added features by extending the base classes.
- Maintainability: The system is easy to maintain and allows adding new information and removing old information.
- Privacy: Users have full authority to the extent they want their information to be displayed to other users and can also block other users.
- Security: It allows only authorized users-transporters, agents or suppliers to access information resources.
- Compatibility: It provides backward and forward compatibility i.e., the system is accessible from older and newer machines.
- Usability: ETransQ is very user friendly.
- Stability: ETransQ has an optimizing code to reduce load on the application server and database server.
- Reusability: The system is inspired by Object orientation methodology, thus enhancing the reusability of the system.

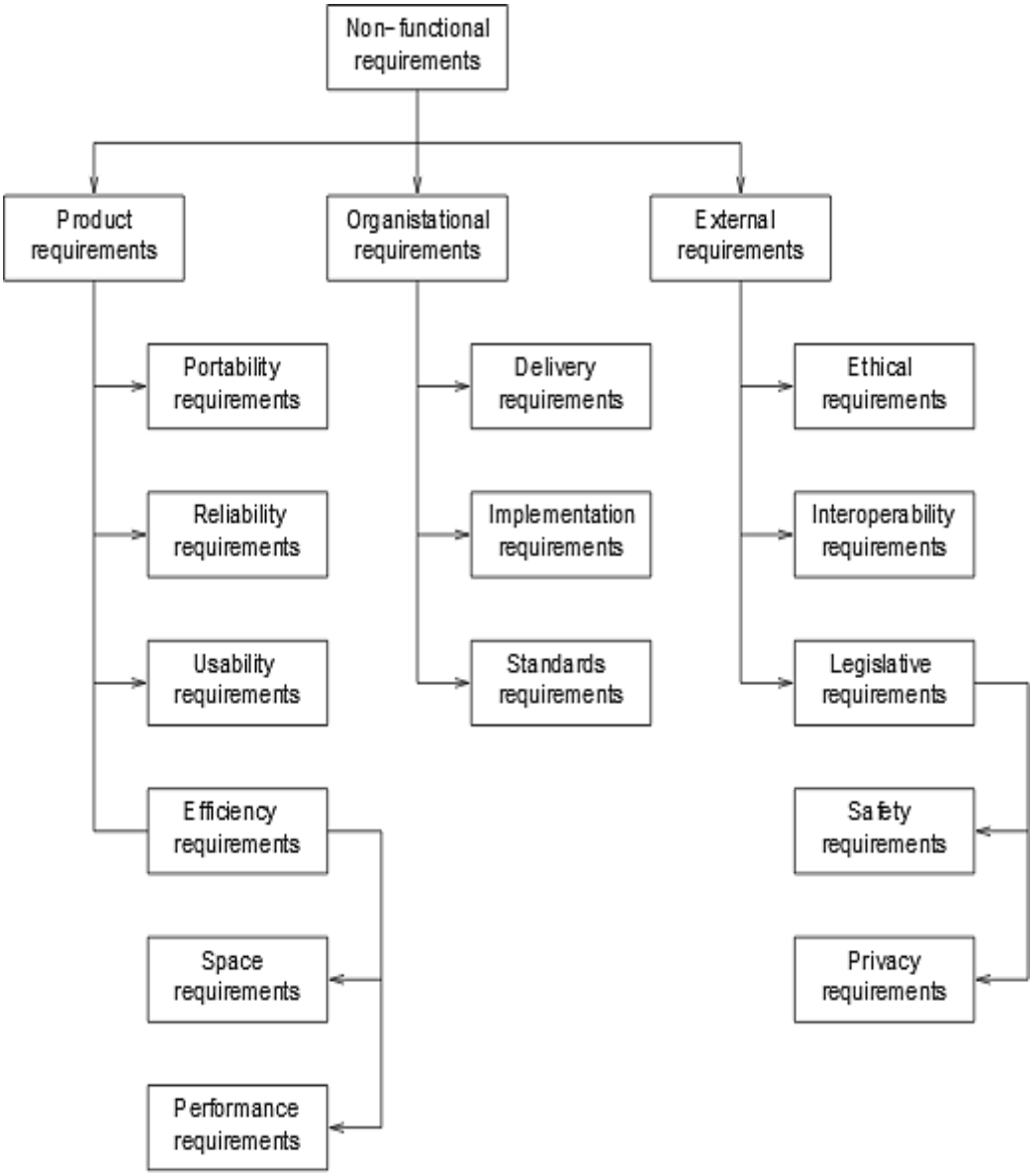


Figure 2.1: Non-functional Requirements

- Robustness: The system being developed consists of validations and checks which will help prevent invalid inputs for smooth functioning of the system.

Domain requirements: Domain requirements are the requirements which are characteristic of a particular category or domain of projects. Domain requirements can be functional or nonfunctional.

Functional Requirements	Non Functional Requirements
A functional requirement defines a system or its component.	A non-functional requirement defines the quality attribute of a software system.
It specifies “What should the software system do?”	It places constraints on “How should the software system fulfill the functional requirements?”
Functional requirement is specified by User.	Non-functional requirement is specified by technical peoples e.g. Architect, Technical leaders and software developers.
It is mandatory.	It is not mandatory.
It is captured in use case.	It is captured as a quality attribute.
Defined at a component level.	Applied to a system as a whole.
Helps you verify the functionality of the software.	Helps you to verify the performance of the software.
Functional Testing like System, Integration, End to End, API testing, etc are done.	Non-Functional Testing like Performance, Stress, Usability, Security testing, etc are done.
Usually easy to define.	Usually more difficult to define.

2.2 Requirement Engineering Process

The process to gather the software requirements from client, analyze and document them is known as requirement engineering. The goal of requirement engineering is to develop and maintain sophisticated and descriptive ‘System Requirements Specification’ document.

Requirement engineering is the key phase in software development which decides what to build and it provides the outline of the quality of the final product. Requirement engineering is the disciplined, process-oriented approach to the development and management of requirements. Requirement Engineering Process is a five step process, which includes,

- Feasibility Study
- Requirement Gathering/ Requirement Elicitation and analysis
- Software Requirement Specification
- Software Requirement Validation
- Requirements Management

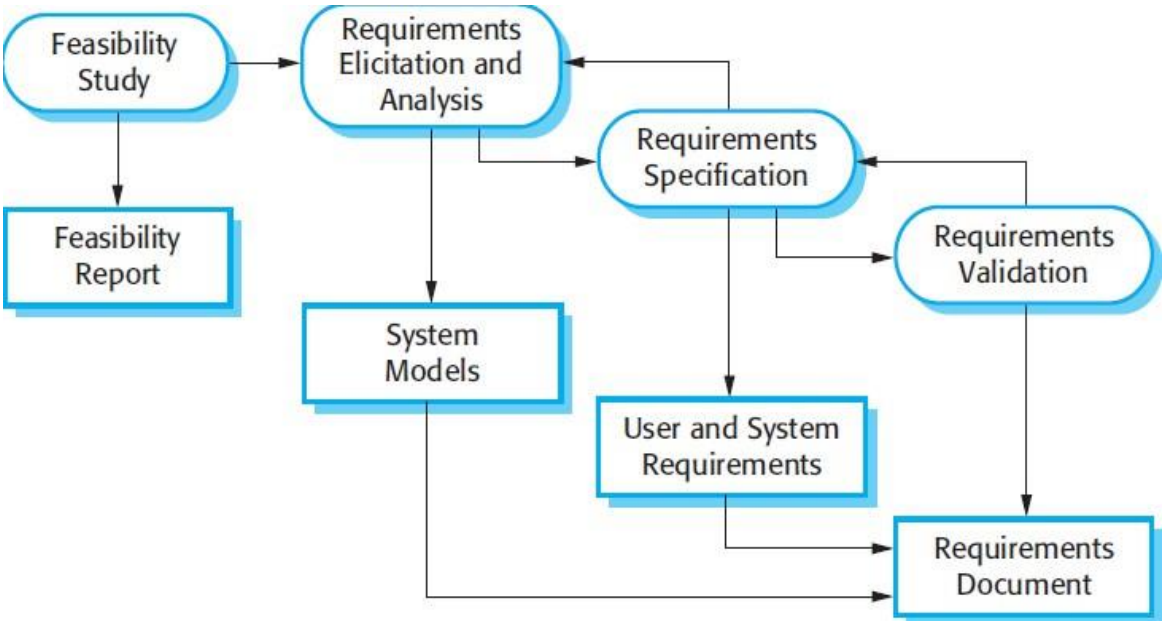


Figure 2.2: Requirement Engineering Process

2.2.1 Feasibility Study

The main aim of a feasibility study is to create reasons for the development of the software that the users accept, that is flexible enough and open to changes, and abide by the standards chosen for software development and maintenance.

There are several types of feasibility. They are:

Technical Feasibility: The current technologies are evaluated using technical feasibility, and they are necessary to achieve the requirements of the customer within the given time and budget.

Operational Feasibility: The assessment of the range for software in which the required software performs a series of levels to solve the problems in business and the customer's requirements.

Economic Feasibility: If the required software can generate profits in the area of finance is decided by economic feasibility.

2.2.2 Requirement Elicitation / Requirement Gathering

If the feasibility report is positive towards undertaking the project, next phase starts with gathering requirements from the user. Requirements Elicitation is the process to find out the requirements for an intended software system by communicating with client, end users, system users and others who have a stake in the software system development.

Requirement elicitation aims to gather requirements from different perspectives to understand the customer needs in the real scenario.

Analysts and engineers communicate with the client and end-users to know their ideas on what the software should provide and which features they want the software to include.

Requirement Elicitation Techniques (Requirement gathering)

There are various ways to discover requirements. The various techniques are,

Interviews

Interviews are strong medium to collect requirements. Organization may conduct several types of interviews such as:

- Structured (closed) interviews, where every single information to gather is decided in advance, they follow pattern and matter of discussion firmly.
- Non-structured (open) interviews, where information to gather is not decided in advance, more flexible and less biased.
- Oral interviews
- Written interviews
- One-to-one interviews which are held between two persons across the table.
- Group interviews which are held between groups of participants. They help to uncover any missing requirement as numerous people are involved.

Surveys

Organization may conduct surveys among various stakeholders by querying about their expectation and requirements from the upcoming system.

Questionnaires

- A document with pre-defined set of objective questions and respective options is handed over to all stakeholders to answer, which are collected and compiled.
- A shortcoming of this technique is, if an option for some issue is not mentioned in the questionnaire, the issue might be left unattended.

Task analysis

Team of engineers and developers may analyze the operation for which the new system is required. If the client already has some software to perform certain operation, it is studied and requirements of proposed system are collected.

Domain Analysis

Every software falls into some domain category. The expert people in the domain can be a great help to analyze general and specific requirements.

Brainstorming

An informal debate is held among various stakeholders and all their inputs are recorded for further requirements analysis.

Prototyping

Prototyping is building user interface without adding detail functionality for user to interpret the features of intended software product. It helps giving better idea of requirements. If there is no software installed at client's end for developer's reference and the client is not aware of its own requirements, the developer creates a prototype based on initially mentioned requirements. The prototype is shown to the client and the feedback is noted. The client feedback serves as an input for requirement gathering.

Observation

Team of experts visits the client's organization or workplace. They observe the actual working of the existing installed systems. They observe the workflow at client's end and how execution problems are dealt. The team itself draws some conclusions which aid to form requirements expected from the software.

2.2.3 Software Requirement Specification

SRS is a document created by system analyst after the requirements are collected from various stakeholders. SRS defines how the intended software will interact with hardware, external

interfaces, speed of operation, response time of system, portability of software across various platforms, maintainability, speed of recovery after crashing, Security, Quality, Limitations etc.

The requirements received from client are written in natural language. It is the responsibility of system analyst to document the requirements in technical language so that they can be comprehended and useful by the software development team.

SRS should come up with following features:

- User Requirements are expressed in natural language.
- Technical requirements are expressed in structured language, which is used inside the organization.
- Design description should be written in Pseudo code.
- Format of Forms and GUI screen prints.
- Conditional and mathematical notations for DFDs etc.

2.2.4 Software Requirement Validation

After requirement specifications are developed, the requirements mentioned in this document are validated. User might ask for illegal, impractical solution or experts may interpret the requirements incorrectly. This results in huge increase in cost if not nipped in the bud. Requirements can be checked against following conditions -

- If they can be practically implemented
- If they are valid and as per functionality and domain of software
- If there are any ambiguities
- If they are complete
- If they can be demonstrated

2.2.5 Requirements management

Requirement management is the process of analyzing, documenting, tracking, prioritizing and agreeing on the requirement and controlling the communication to relevant stakeholders. This stage takes care of the changing nature of requirements.

It should be ensured that the SRS is as modifiable as possible so as to incorporate changes in requirements specified by the end users at later stages too. Being able to modify the software as per requirements in a systematic and controlled manner is an extremely important part of the requirements engineering process.

2.3 Software Requirement Specification

A software requirements specification (SRS) is a document explaining how and what the software/system will do. It defines the features and functionality that the product requires to satisfy all stakeholders' (business, users) needs.

A standard SRS includes:

- A goal/purpose
- A summary of the whole process
- Specific Requirements

<u>Requirements document structure</u>

The IEEE standard suggests the following standard:

- Introduction
- Glossary
- User requirements definition
- System architecture
- System requirements specification
- System models
- System evolution
- Appendices and
- Index

SRS Contains the following

Introduction

- **Purpose of this Document –** At first, main aim of why this document is necessary and what's purpose of document is explained and described.
- **Scope of this document –** In this, overall working and main objective of document and what value it will provide to customer is described and explained. It also includes a description of development cost and time required.
- **Overview –** In this, description of product is explained. It's simply summary or overall review of product.

General description

In this, general functions of product which includes objective of user, a user characteristic, features, benefits, about why its importance is mentioned. It also describes features of user community.

Functional Requirements

In this, possible outcome of software system which includes effects due to operation of program is fully explained. All functional requirements which may include calculations, data processing, etc. are placed in a ranked order. Functional requirements specify the expected behavior of the system-which outputs should be produced from the given inputs.

Interface Requirements

In this, software interfaces which mean how software program communicates with each other or users either in form of any language, code, or message are fully described and explained. Examples can be shared memory, data streams, etc.

Performance Requirements

It explains required time, required memory, maximum error rate, etc. The performance requirements part of an SRS specifies the performance constraints on the software system. All the requirements relating to the performance characteristics of the system must be clearly specified. There are two types of performance requirements: static and dynamic.

i. Static requirements are those that do not impose constraint on the execution characteristics of the system.
ii. Dynamic requirements specify constraints on the execution behaviour of the system.

Design Constraints

In this, constraints means limitation or restriction are specified and explained for design team. There are a number of factors in the client's environment that may restrict the choices of a designer leading to design constraints such factors include standards that must be followed resource limits, operating environment, reliability and security requirements and policies that may have an impact on the design of the system. An SRS should identify and specify all such constraints.

Non-Functional Attributes

In this, non-functional attributes are explained that are required by software system for better performance. An example may include Security, Portability, Reliability, Reusability,

Application compatibility, Data integrity, Scalability capacity, etc.

Preliminary Schedule and Budget

In this, initial version and budget of project plan are explained which include overall time duration required and overall cost required for development of project.

Appendices

In this, additional information like references from where information is gathered, definitions of some specific terms, acronyms, abbreviations, etc. are given and explained.

SRS Template

The following is a simple SRS template:

Table of Contents

1. Introduction

1.1 Purpose of this document

1.2 Scope of this document

1.3 Overview

1.4 Business Context

2. General Description

2.1 Product Functions

2.2 Similar System Information

2.3 User Characteristics

2.4 User Problem Statement

2.5 User Objectives

2.6 General Constraints

3. Functional Requirements

4. Interface Requirements

4.1 User Interfaces

4.2 Hardware Interfaces

4.3 Communications Interfaces

4.4 Software Interfaces

5. Performance Requirements

6. Other non-functional attributes

6.1 Security

6.3 Reliability

6.4 Maintainability

6.5 Portability

6.6 Extensibility

6.7 Reusability

6.8 Application Affinity/Compatibility

7. Operational Scenarios

8. Preliminary Use Case Models and Sequence Diagrams

8.1 Use Case Model

8.2 Sequence Diagrams

9. Updated Schedule

10. Appendices

10.1 Definitions, Acronyms, Abbreviations

10.2 References

Features of an SRS

An SRS should have following characteristics:

Correct - should accurately reflect product functionality and specification at any point of time.

Unambiguous - should not be any confusion regarding interpretation of the requirements.

Complete - should contain all the features requested by a client.

Consistent - same abbreviation and conventions must be followed throughout the document.

Ranked for importance and/or stability - every requirement is important. But some are urgent and must be fulfilled before other requirements and some could be delayed. It's better to classify each requirement according to its importance and stability.

Verifiable - an SRS is verifiable only if every stated requirement can be verified. A requirement is verifiable if there is some method to quantifiably measure whether the final software meets that requirement.

Modifiable -- an SRS must clearly identify each and every requirement in a systematic manner. If there are any changes, the specific requirements and the dependent ones can be modified accordingly without impact the others.

Traceable – an SRS is traceable if the origin of each of its requirements is clear and if it makes it easy to reference each requirement in future development.

Example: SRS for Attendance Maintaneence system

Software Requirement Specification for Attendance maintenance System

Prepared by Pushpalatha

Decembner 21,2023

Release 1.0

Version 1.0

Table of Contents

1. INTRODUCTION 1
1.1 PURPOSE 1
1.2 SCOPE 1
1.3 DEFINITIONS, ACRONYMS, AND ABBREVIATION 1
1.4 REFERENCE 2
1.5 OVERVIEW 2
2. GENERALDESCRIPTION 2
2.1 PRODUCT PERSPECT 2
2.2 PRODUCT FUNCTIONS 2
2.3 USER CHARACTERISTICS 3
2.4 ASSUMPTIONS AND DEPENDENCIES 3
3. SPECIFIC REQUIREMENTS 4
3.1 EXTERNAL INTERFACE REQUIRE 4
3.1.1 User Interfaces 4
3.1.2 Hardware Interfaces 4
3.1.3 SoftwareInterfaces 4
3.1.4 Communications Interfaces 4
3.2 FUNCTIONAL REQUIREMENTS 5
3.2.1 Student Registration 5

3.2.2 Student Attendance Management 5
3.2.3 Opinion Management 5
3.3 USECASES..... 5
3.4 CLASSES / OBJECTS DIAGRAM.....6
3.5 NON-FUNCTIONAL REQUIREMENTS 7
3.5.1 Performance 7
3.5.2 Reliability.....7
3.5.3 Availability.....7
3.5.4 Security 8
3.5.5 Maintainability 8
3.5.6 Portability.....8
3.6 DESIGN CONSTRAINTS 8
3.7 OTHER REQUIREMENTS..... 8
4. ANALYSIS MODELS..... 9
4.1 SEQUENCE DIAGRAMS 9
A. APPENDICES

1. Introduction

Student management system has become important factors in modern education field. This system should help the institutional to streamline the administrative task and provide real-time access to the data. Building this system in web based interface will further help the ease of accessibility through any web browser. The study findings enable the definition of the project problem statement, its objectives, scopes and advantages of the student management system.

1.1 Purpose

The purpose this documents is to present a detailed description of the Student Management System. It will explain the purpose and features of the software, the interfaces of the software, what the software will do, the constraints under which it must operates and how the software

will react to external stimuli. This document is intended for both the end users and the developers of the software.

1.2 Scope

This document covers the requirements for the Student Management System. This software will provide a graphical environment in which the users of the system will be able to perform various operations that are associated with storing, marinating, updating and retrieving Student information. The purpose of this is to guide developers in selecting a design that will be able to accommodate the full-scale application.The system will capture information about student's personal details lectures and the courses. Storing updating and retrieving in a fast and accurate way.

1.3 Definitions, Acronyms, and Abbreviations

The Student Management System has to handle records for many number of students and maintenance was difficult. Though it has used an information system, it was totally manual. Hence there is a need to upgrade the system with a computer based information system.

1.4 References

An Integrated Approach to Software Engineering Approach - Pankaj Jalote Software Engineering A Practitioner's Approach - Roger S Pressman

1.5 Overview

The purpose this documents is to present a detailed description of the Student Management System. It will explain the purpose and features of the software, the interfaces of the software, what the software will do, the constraints under which it must operates and how the software will react to external stimuli. This document is intended for both the end users and the developers of the software.

2. General Description

2.1 Product Perspective

The product Student Management system, is an independent product and does not depend on any other product or system. The product will automate various tasks associated with handling student details and better organizing the stored information and optimum performance, thus helping the Colleges to ensure smooth working of these processes.

2.2 Product Functions

Our system has two types of accessing modes,

1. Administrator
2. User
    - 2.1 Teacher
    - 2.2 Student

i) Administrator:

SMS is managed by Administrator. Administrator has to update and monitor the registered student details, add a new student, provide register number for all students, assign each student a course etc., Administrator can update his profile, and also can give help to the teachers and students.

ii) User:

There are two users:

a. Student:User can only view their personal details, course assigned, and edit their assigned course and can view their attendance.

b. Teacher: User can add them onto the portal and view their schedules, marks attendance of the students, also can view the students details in graphical order, also of a single student and about the views from the students.

2.3 User Characteristics

This software gives access to two kinds of users.

1. Administrator: The personnel and College administrator will have administrator access to add, delete and modify information stored in the database.

2. Authorized User: Teaching staff will have access to only view the data stored in the database and can update the student's attendance in the form of formatted reports.

2.4 Assumptions and Dependencies

- We assume that the Office personnel do all the data entry based and the correct values obtained from forms and registers.
- We assume that the computers that will use the software will be part of the college LAN.

- Users with administrator access should be careful in deleting or modifying any information knowingly or unknowingly which will lead to inconsistency of the database.
- The end users of this software are assumed to have basic level of computer knowledge i.e. point and click.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

- GUI along with meaningful Frames and buttons
- Reports are generated as per the requirement

3.1.2 Hardware Interfaces

Hardware Environment	Dual Core 2nd generation/
System Configuration	RAM-512 MB HDD-80GB
Operating system	Windows XP/Vista/7/8/8.1

3.1.3 Software Interfaces

Front End	PHP 5.3.0
Back End	MySQL 5.1.36

When invalid inputs are given to the modules then the error messages will be popped up in order to inform the user that the input provided is not taken by the database. When incomplete information is provided by the user and the user tries to submit the form in order to store the details in the database the system will pop up a message box asking the user to enter all the details required.

3.1.4 Communications Interfaces

The machine will have to be part of the college Local area Network to access the central database.

3.2 Functional Requirements

Student Management System involves the following functions

3.2.1 Student Registration

- SMS provides online registration and status information to the student to view their status.
- SMS provides automatic student register number generation based on course and year.
- SMS provides to students to add them in their course they want to study.

3.2.2 Student Attendance Management

- Easily track attendance information of students.
- Quickly produce single or multiple day attendance bulletins.

3.2.3 Opinion Management

- SMS provides a comprehensive opinion scheduling based on course.
- Students can facilitate to give their opinions by giving the teacher rank

3.3 Use cases

Following are the purposes of a use case diagram given below:

- It gathers the system's needs.
- It depicts the external view of the system.
- It recognizes the internal as well as external factors that influence the system.

It represents the interaction between the actors.

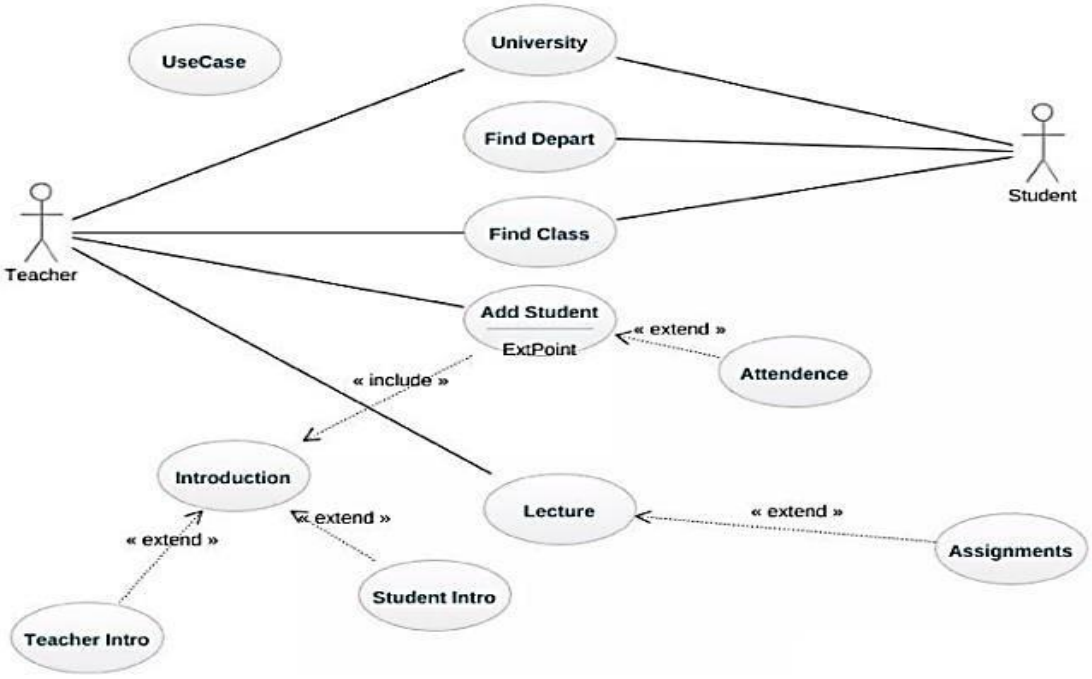
3.4 Classes / Objects Diagram

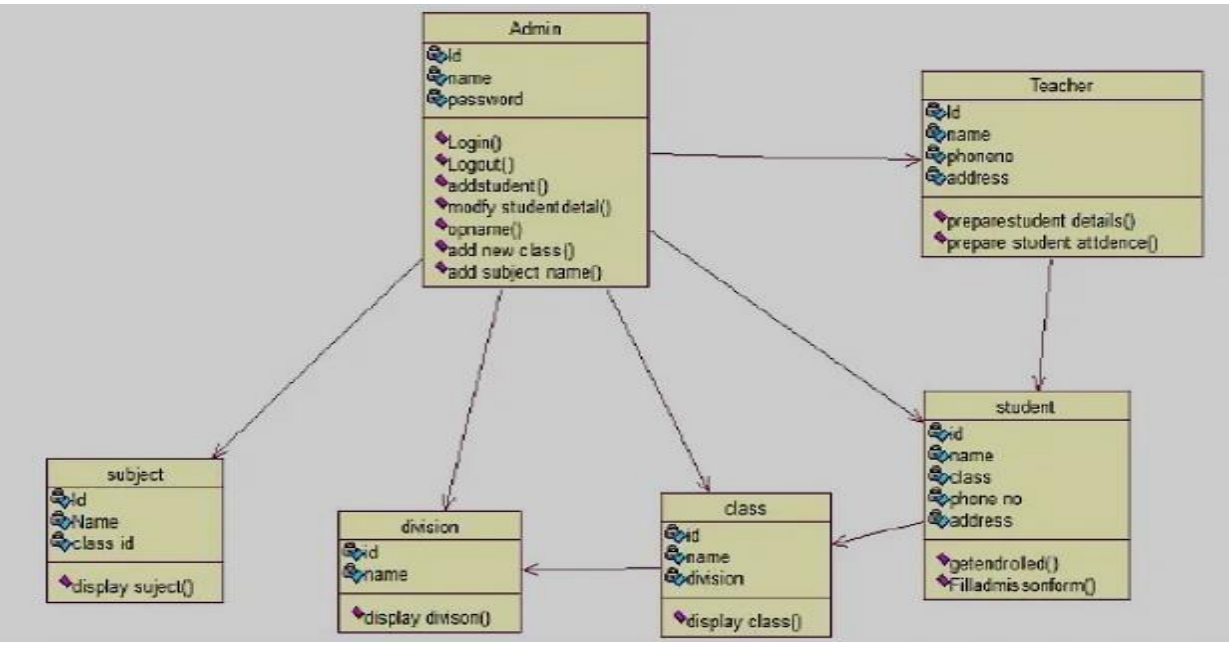
Class diagram shows a collection of classes, interfaces, associations, collaborations, and constraints. It is also known as a structural diagram.

The purpose of class diagram is to model the static view of an application. Class diagrams are theonly diagrams which can be directly mapped with object-oriented languages and thus widely used at the time of construction.

The purpose of the class diagram can be summarized as

- Analysis and design of the static view of an application
- Describe responsibilities of a system.
- Base for component and deployment diagrams.
- Forward and reverse engineering





3.5 Non-Functional Requirements

3.5.1 Performance

Easy tracking of records and updating can be done. All the requirements relating to performance characteristics of the system are specified in the section below. There are

two types of requirements.

A. Static Requirements

These requirements do not impose any constraints on the execution characteristics of the system. They are:

1) Number of Terminals:

The software makes use of an underlying database that will reside at the server, while the front end will be available online to the administrative and departmental computers as well as students and teachers.

2) Number of Users:

The number of users may vary, as this software finds applications in almost all department of the organization.

B. Dynamic Requirements

These specify constraints on the execution characteristics of the system. They typically include response time and throughout of the system. Since these factors are not applicable to the proposed software, it will suffice if the response tine is high and the transactions are carried out precisely and quickly.

3.5.2 Reliability

The software will not be able to connect to the centralized database in the event that the college LAN fails or in the event of the server being down due to a hardware or software failure.

3.5.3 Availability

The software will be available only to authorized users of the colleges like teachers to mark the students attendance, student to view their enrolled course, admin to add an update students records.

3.5.4 Security

The security requirements deal with the primary security. The software should be handled only by the administrator and authorized users. Only the administrator has right to assign permission like creating new accounts and generating password. Only authorized users can access the system with username and password.

3.5.5 Maintainability

Backups for database are available.

3.5.6 Portability

The Software is a web-based application and is built in PHP and MYSQL so it is platform independent and is independent of operating system.

3.6 Design Constraints

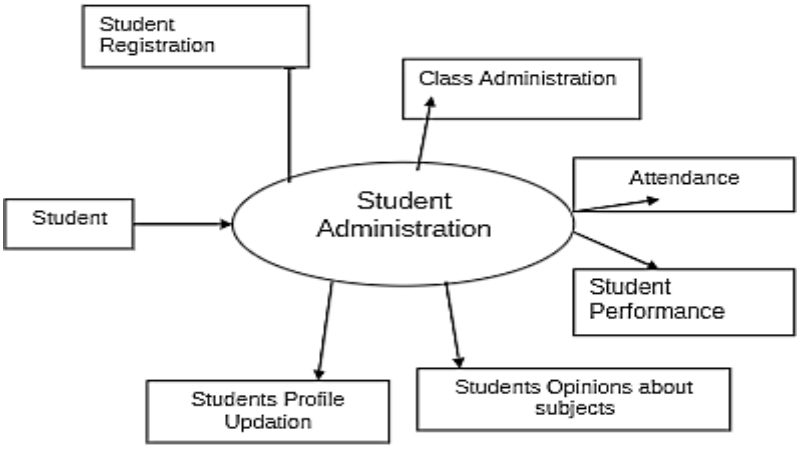
- This software provides security.
- The login form prevents the system from being misused by unauthorized users.
- Only an authorized operator will be granted rights to modify as per requirements. This software is also reliable and fault tolerant.
- The system developed is designed to handle invalid inputs.
- Since reliability is major area of concern the system has a backup to avoid data loss.
- The user should know the programming language very well that is used to develop a software.

3.7 Other Requirements

- Database
- All the data will be stored in a relational database

4. Analysis Models

4.1 Sequence Diagrams



2.4 Formal Specification

Formal specifications are mathematically based techniques whose purpose are to help with the implementation of systems and software. They are used to describe a system, to analyze its behavior, and to aid in its design by verifying key properties of interest through rigorous and

effective reasoning tools. These specifications are *formal* in the sense that they have a syntax, their semantics fall within one domain, and they are able to be used to infer useful information.

A good specification will have:

- Constructability, manageability
- Usability
- Communicability
- Powerful and efficient analysis

2.4.1 Formal specification in the software process

Both the system requirements and the system design are expressed in detail and carefully analysed and checked before implementation begins. If a formal specification of the software is developed, this usually comes after the system requirements have been specified but before the detailed system design. There is a tight feedback loop between the detailed requirements specification and the formal specification.

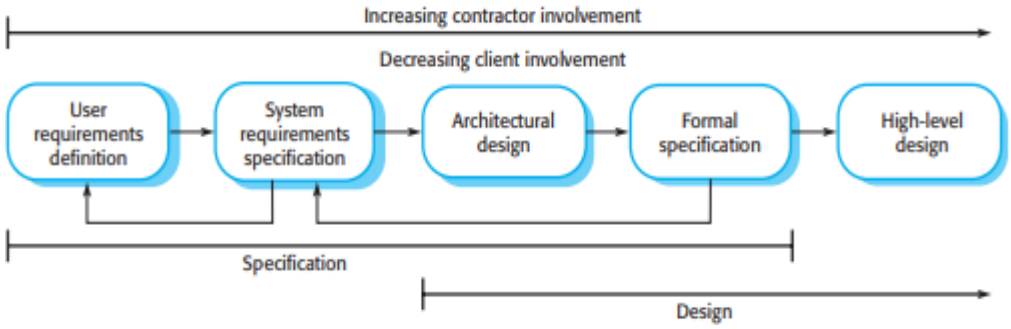


Figure 2.3: Specification and Design

The involvement of the client decreases and the involvement of the contractor increases as more detail is added to the system specification. In the early stages of the process, the specification should be ‘customer-oriented’. You should write the specification so that the client can understand it, and you should make as few assumptions as possible about the software design. However, the final stage of the process, which is the construction of a complete, consistent and precise specification.

Figure a shows the stages of software specification and its interface with the design process. There is a two-way relationship between each stage in the process. Information is fed from the specification to the design process and vice versa

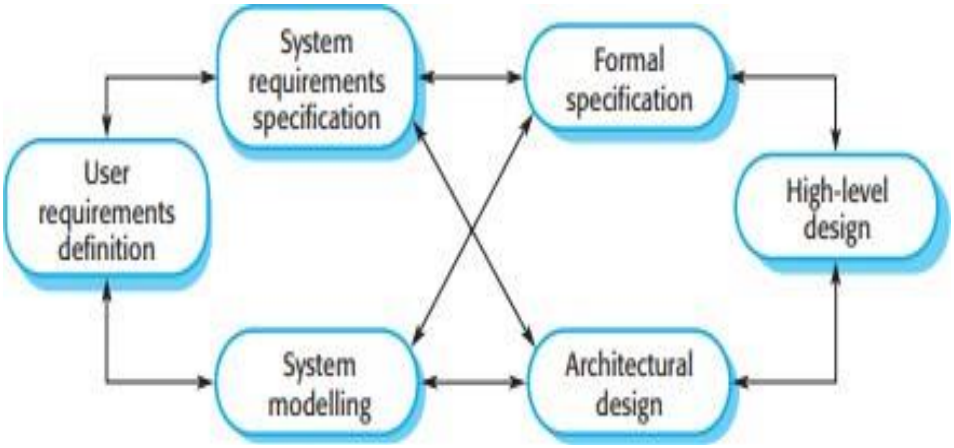


Figure 2.4: Formal specification in the software process

. As you develop the specification in detail, your understanding of that specification increases. Creating a formal specification forces you to make a detailed systems analysis that usually reveals errors and inconsistencies in the informal requirements specification.

Two fundamental approaches to formal specification are

1. An algebraic approach where the system is described in terms of operations and their relationships.
2. A model-based approach where a model of the system is built using mathematical constructs such as sets and sequences and the system operations are defined by how they modify the system state

Formal Specification language

	Sequential	Concurrent
Algebraic	Larch (Guttag, *et al.*, 1993), OBJ (Futatsugi, *et al.*, 1985)	Lotos (Bolognesi and Brinksma, 1987),
Model-based	Z (Spivey, 1992) VDM (Jones, 1980) B (Wordsworth, 1996)	CSP (Hoare, 1985) Petri Nets (Peterson, 1981)

2.4.2 Sub-system interface specification

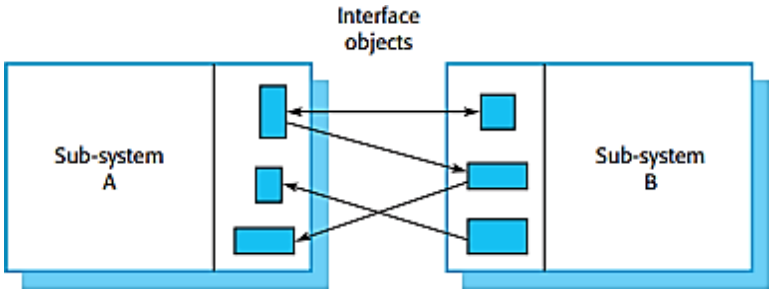


Figure 2.5: Sub-system interface objects

Sub-system interface specification Large systems are usually decomposed into sub-systems that are developed independently. Sub-systems make use of other sub-systems, so an essential part of the specification process is to define sub-system interfaces.

The process of developing a formal specification of a sub-system interface includes the following activities:

1. Specification structuring Organize the informal interface specification into a set of abstract data types or object classes. You should informally define the operations associated with each class.

2. Specification naming Establish a name for each abstract type specification, decide whether or not they require generic parameters and decide on names for the sorts identified.

3. Operation selection Choose a set of operations for each specification based on the identified interface functionality. You should include operations to create instances of the sort, to modify the value of instances and to inspect the instance values. You may have to add functions to those initially identified in the informal interface definition.

4. Informal operation specification Write an informal specification of each operation. You should describe how the operations affect the defined sort.

5. Syntax definition Define the syntax of the operations and the parameters to each operation. This is the signature part of the formal specification. You should update the informal specification at this stage if necessary.

6. Axiom definition Define the semantics of the operations by describing what conditions are always true for different operation combinations.

2.5 Finite state machines

- The finite Automaton is a mathematical model of a system with discrete inputs and outputs.

- The system can be in a any one of a finite number of internal configuration or states.
- The state of the system summarizes the information concerning past inputs that is needed to determine the behavior of the system on subsequent inputs.
- The primary example of finite automaton is a switching circuit such as control unit of a computer.
- A switching circuit is composed of a finite number of gates each of which can be in one of two conditions usually 0 and 1

2.5.1 Deterministic finite automata

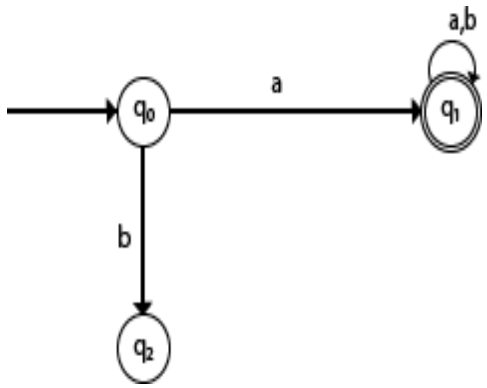
- A finite automaton (FA) consists of a finite set of states and a set of transitions from state to state that occur on an input symbol chosen from an alphabet
- For each input symbol there is exactly one transition out of each state.
- One state denoted qo is the initial state in which the transition starts.
- Some states are designated as final states or accepting states.
- The Deterministic finite automata can be represented as a transition diagram or transition table.
- The directed graph is used to represent the transition diagram.

  i. The vertices of the graph correspond to the states of FA

  ii. If there is a transition from state q to state p on input a there is an are labeled a from state q to state p

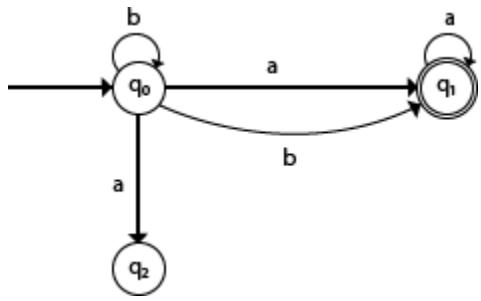
  iii. The FA accepts a string x if the sequence of symbols of x leads start state to final state.

Example:



2.5.2 Non – deterministic finite automata

- A finite automata model to allow zero, one or more transitions from a state on the same input sympol. This new model is called a non deterministic finite Automata (NFA)
- Any set accepted by NFA can also be accepted by DFA.
- The input sequence a1, a2, a3 …. an is accepted by NFA, if the transition leads from the initial state of final state.



2.6 Petri nets

Major difficulty with specifying concurrent systems is coping with timing. Timing problems arise as a consequence of a poor design or a faulty implementation. One powerful technique for specifying systems with potential timing problems is Petri Nets.

- A Petri net (also known as a place / transition net or P / T net) is one of several mathematical modeling languages for the decryption of distributed System.
  - A Petri net is a directed bipartite graph, in which the nodes represent transitions (i.e. events that may occur, signified by bars) and places (i.e. conditions, signified by circles). The directed arcs describe which places are pre – and / or post conditions for which transitions (signified by arrows). It was first introduced by Carl Adam Petri in 1962. It is a diagrammatic tool to model concurrency and synchronization in distributed systems.

A Petri net consists of four parts:

- A set of places P
- A set of transitions T
- An input function I
- An output function O

a) Set of places:

P is {p1, p2, p3, p4}

b) Set of transitions:

T is {t1, t2}

c) Input function:

I (t1) = {P2, P4}

I (t2) = {P2}

d) Output function:

O (t1) = {P1}

O (t2) = {P3, P3}

More formally, a Petri net is a 4 – tuple C = (P, T, I ,O)

Where,

- P = {P1, P2, P3,. ..... , Pn} is a finite set of places,
- n >= 0

T = {t1,t2,t3,..... tn} is a finite set of transaction, m >= 0 with P and T disjoint.

I: T -> P is the Input function, a mapping from transitions to bags of places.

O: T -> P is the Output function, a mapping from transitions to bags of places.

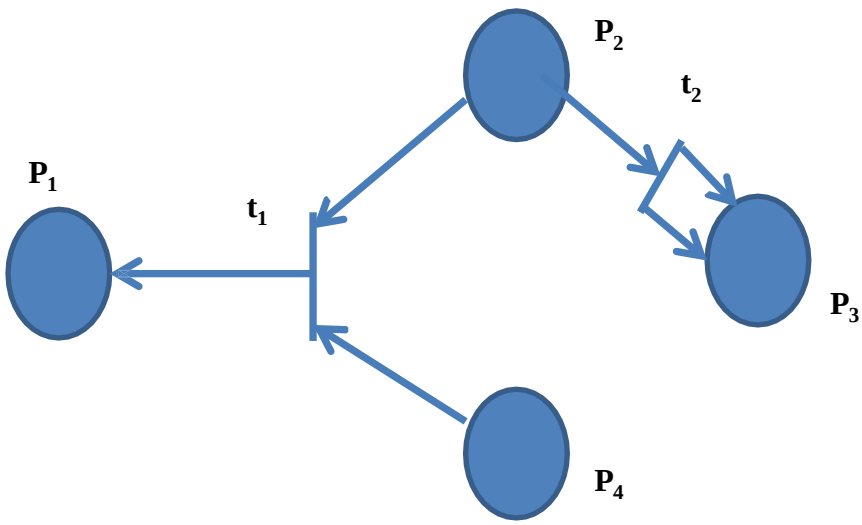
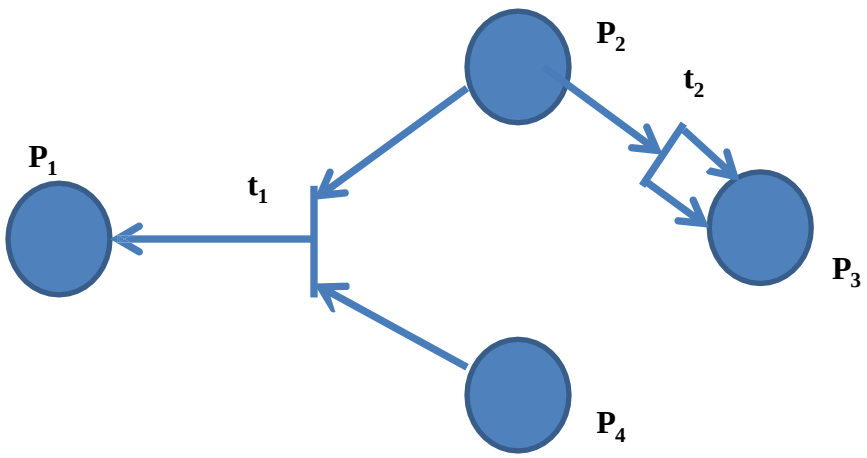


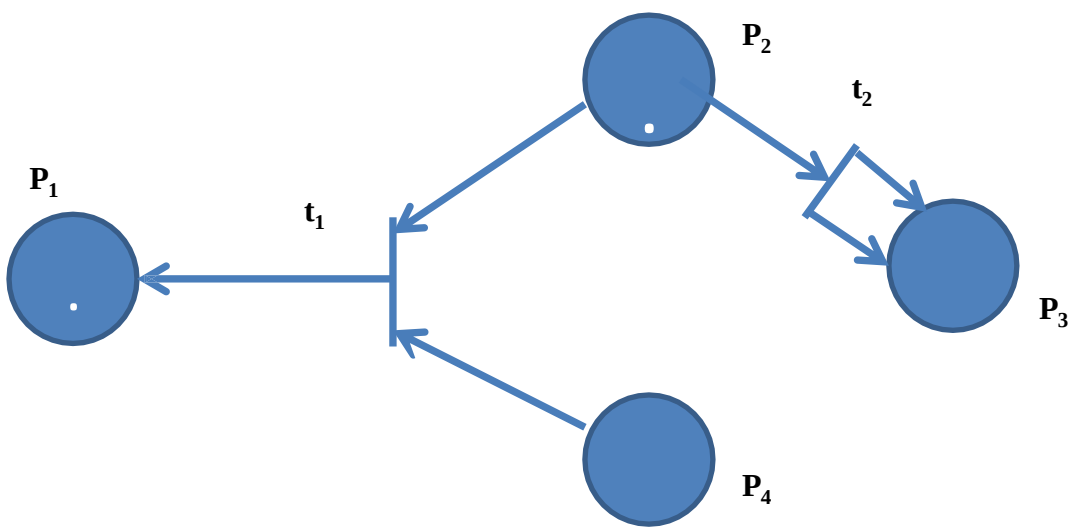
Figure 2.6: Petri Nets

- A marking of a Petri net is an assignment of tokens to that Petri net. In the below figure, there are four tokens: one in P1, two in P2, none in P3, and one in P4.
- They are represented by the vector (1, 2, 0, and 1).

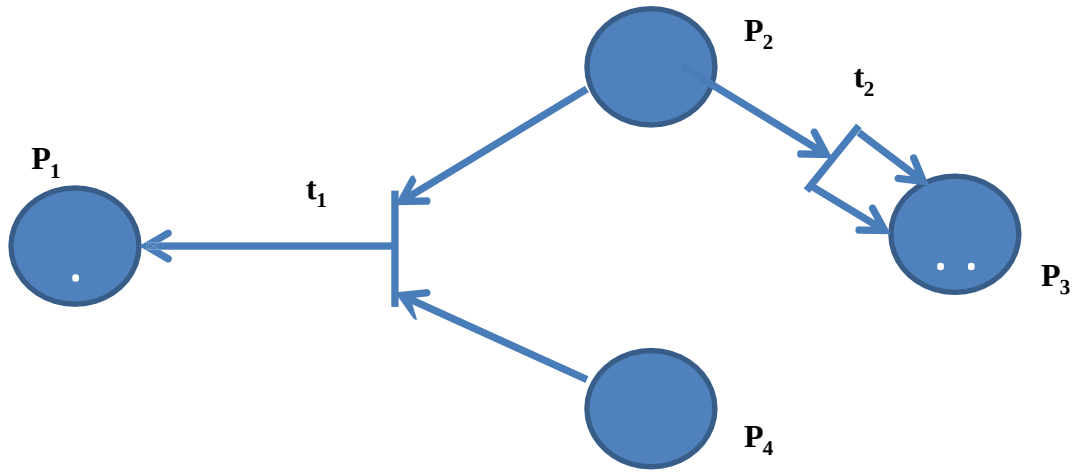
- A transition is enabled if each of its input places has as many tokens in it as there are arcs from the place to that transition. Transition t1 is enabled (ready to fire).
- If t1 fires one token is removed from P2, and one from P4, and one new token is placed in P1.



- Transition t2 is also enabled. The number of tokens in not conserved
- Petri nets are indeterminate
    - Suppose t1 fires
    - The resulting marking is (2,1,0,0)



- Now only t2 is enabled
- It fires



- The marking is now (2, 0, 2, 0)

More formally, a marking M of a Petri net C = (P, T, I, O) is a function from the set of places P to the non – negative integers,

M : P -> {0,1,2, ……}

A marked Petri net is then a 5 – tuple (P, T, I, O, M)

<u>Inhibitor arcs</u>

An inhibitor arc is marked by a small circle, not an arrowhead

- Transition t1 is enabled

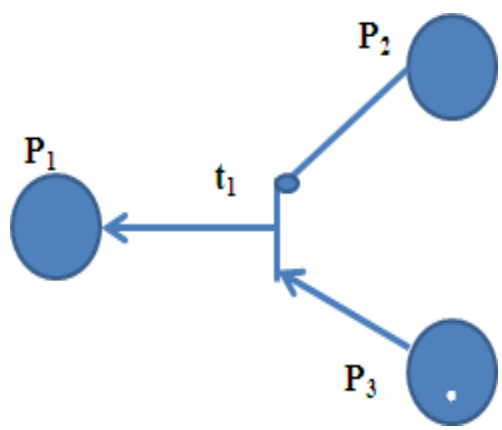


Figure 2.7: Petri net with an inhibitor arcs

In general, a transition is enabled if there is at least one token on each (normal) input are, and no tokens on any inhibitor input arcs. This extension is used in the petri net specification of the elevator problem.

2.7 Object Modeling using UML

Object is a combination of data and logic that represents some real world entity. Data are called attributes ,logic is called as functions

Window
Size Color
Open() Close()

Figure 2.8: Object representation

. Example: car(object)

- Attributes:car name,color,number of doors
- Functions: stop, change,milage

Analysis and Design

i. Analysis emphasizes an investigation of the problem and requirements, rather than solution.ie)do the right thing

ii. Design emphasizes a conceptual solution that fulfills the requirements, rather than its implementation, do the thing right

What is OOAD

- Object-oriented analysis there emphasis on finding and describing the objects or concepts in the problem domain.
- For example, in the case of the flight information system, some of the concepts include Plane, Flight, and Pilot.
- During object-oriented design (or simply, object design) there is an emphasis on defining software objects and how they collaborate to fulfill the requirements.
- For example, a Plane software object may have a tail Number attribute and a getFlightHistory method.

2.8 Unified Modeling Language

Unified Modeling Language (UML) is a general-purpose modeling language. The main aim of UML is to define a standard way to **visualize** the way a system has been designed. It is quite

similar to blueprints used in other fields of engineering. UML is **not a programming language**, it is rather a visual language.

UML Diagrams

- The Unified Modeling Language is a visual language for specifying, constructing and documenting the artifacts of systems.
- Unified Modeling Language (UML) is a standardized general-purpose modeling language in the field of software engineering.
- The standard is managed, and was created by, the Object Management Group. UML includes a set of graphic notation techniques to create visual models of software-intensive systems.

There are two broad categories of diagrams and they are again divided into subcategories −

i. Structural Diagrams

ii. Behavioral Diagrams

2.8.1 Structural Diagrams

- The structural diagrams represent the static aspect of the system. These static aspects represent those parts of a diagram, which forms the main structure and are therefore stable.
- These static parts are represented by classes, interfaces, objects, components, and nodes. The four structural diagrams are

a) Class diagram
b) Object diagram
c) Component diagram
d) Deployment diagram

a) Class Diagram

- Class diagrams are the most common diagrams used in UML. Class diagram consists of classes, interfaces, associations, and collaboration.
- Class diagrams basically represent the object-oriented view of a system, which is static in nature.
- Active class is used in a class diagram to represent the concurrency of the system.
- Class diagram represents the object orientation of a system.

- Hence, it is generally used for development purpose. This is the most widely used diagram at the time of system construction.

b) Object Diagram

- Object diagrams can be described as an instance of class diagram. Thus, these diagrams are more close to real-life scenarios where we implement a system.
- Object diagrams are a set of objects and their relationship is just like class diagrams. They also represent the static view of the system.
- The usage of object diagrams is similar to class diagrams but they are used to build prototype of a system from a practical perspective.

c) Component Diagram

- Component diagrams represent a set of components and their relationships. These components consist of classes, interfaces, or collaborations.
- Component diagrams represent the implementation view of a system.
- During the design phase, software artifacts (classes, interfaces, etc.) of a system are arranged in different groups depending upon their relationship.
- Now, these groups are known as components.
- Finally, it can be said component diagrams are used to visualize the implementation.

e) Deployment Diagram

- Deployment diagrams are a set of nodes and their relationships. These nodes are physical entities where the components are deployed.
- Deployment diagrams are used for visualizing the deployment view of a system. This is generally used by the deployment team.

2.8.2 Behavioral Diagrams

- Any system can have two aspects, static and dynamic. So, a model is considered as complete when both the aspects are fully covered.
- Behavioral diagrams basically capture the dynamic aspect of a system. Dynamic aspect can be further described as the changing/moving parts of a system.

UML has the following five types of behavioral diagrams −

a) Use case diagram
b) Sequence diagram

c) Collaboration diagram
d) State chart diagram
e) Activity diagram

a) Use Case Diagram

- Use case diagrams are a set of use cases, actors, and their relationships. They represent the use case view of a system.
- A use case represents a particular functionality of a system.
- Hence, use case diagram is used to describe the relationships among the functionalities and their internal/external controllers. These controllers are known as actors.

b) Sequence Diagram

- A sequence diagram is an interaction diagram.
- From the name, it is clear that the diagram deals with some sequences, which are the sequence of messages flowing from one object to another.
- Interaction among the components of a system is very important from implementation and execution perspective.
- Sequence diagram is used to visualize the sequence of calls in a system to perform a specific functionality.

c) Collaboration Diagram

- Collaboration diagram is another form of interaction diagram. It represents the structural organization of a system and the messages sent/received.
- Structural organization consists of objects and links.
- The purpose of collaboration diagram is similar to sequence diagram.
- However, the specific purpose of collaboration diagram is to visualize the organization of objects and their interaction.

d) State chart Diagram

- Any real-time system is expected to be reacted by some kind of internal/external events. These events are responsible for state change of the system.
- State chart diagram is used to represent the event driven state change of a system. It basically describes the state change of a class, interface, etc.
- State chart diagram is used to visualize the reaction of a system by internal/external factors.

e) Activity Diagram

- Activity diagram describes the flow of control in a system. It consists of activities and links. The flow can be sequential, concurrent, or branched.
- Activities are nothing but the functions of a system. Numbers of activity diagrams are prepared to capture the entire flow in a system.
- Activity diagrams are used to visualize the flow of controls in a system. This is prepared to have an idea of how the system will work when executed.

2.9 Use cases

Use cases are text stories widely used to discover and record requirements. They are text stories of some actor using a system to meet goals.

Example: Process sale

A customer arrives at a checkout with items to purchase. The cashier uses the POS system to record each purchased item. The system presents a running total and line-item details. The customer enters payment information, which the system validates and records. The system updates inventory. The customer receives a receipt from the system and then leaves with the items.

2.9.1 Actors, Scenarios, and Use Cases

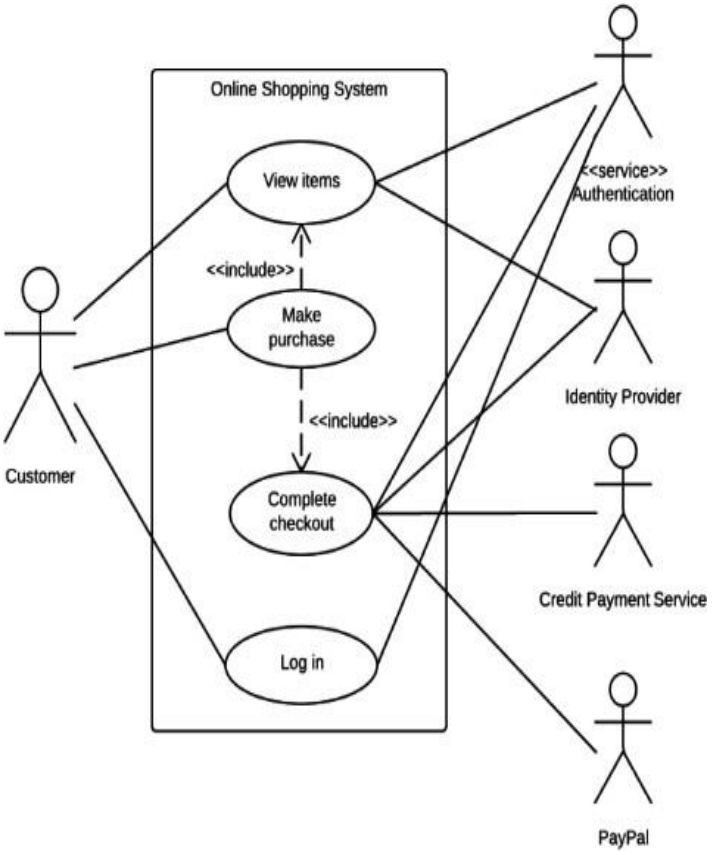
Actor: an actor is something with behavior, such as a person (identified by role), computer system, or organization; for example, a cashier.

A scenario is a specific sequence of actions and interactions between actors and the system; it is also called a use case instance.

- It is one particular story of using a system, or one path through the use case; for example, the scenario of successfully purchasing items with cash, or the scenario of failing to purchase items because of a credit payment denial.
- Informally then, a use case is a collection of related success and failure scenarios that describe an actor using a system to support a goal.

For example, here is a casual format use case with alternate scenarios:Handle Returns

Main Success Scenario: A customer arrives at a checkout with items to return. The cashier uses the POS system to record each returned item



Alternate Scenarios:

If the customer paid by credit, and the reimbursement transaction to their credit account is rejected, inform the customer and pay them with cash.If the item identifier is not found in the system, notify the Cashier and suggest manual entry of the identifier code (perhaps it is corrupted).If the system detects failure to communicate with the external accounting system.

2.9.2 Use case Model

Use case model is the set of all written use cases; it is a model of the system's functionality and environment**.**

- Use cases are text documents, not diagrams, and use-case modeling is primarily an act of writing text, not drawing diagrams**.**

2.9.3 Kinds of actors

1. Primary actor

Primary actor has user goals fulfilled through using services of the SuD. For example, the cashier.

2. Supporting actor

- Supporting actor provides a service (for example, information) to the SuD.
- The automated payment authorization service is an example. Often a computer system, but could be an organization or person.

3. Offstage actor

Offstage actor has an interest in the behavior of the use case, but is not primary or supporting; for example, a government tax agency.

2.9.4 Three Common Use Case Formats

1. brief- one paragraph summary, usually of the main success scenario.
2. casual- Informal paragraph format. Multiple paragraphs that cover various scenarios.
3. fully dressed- All steps and variations are written in detail, and there are supporting sections, such as preconditions and success guarantees.

2.9.6 Use case section

Use case section	**Comment**
Use case name	Start with verb
Scope	The system under design
Level	“user goal “or “sub function”
Primary actor	Calls on the system to deliver its services
Stack holder&Interest	Who cares about this use case and what do they want
Preconditions	What must be true on start, and worth telling the reader
Success Guarantee	What must be true on successful completion and worth telling the reader
Main success scenario	Unconditional happy path scenario of success
Extensions	Alternate scenarios of success or failure
Special requirements	Related nonfunctional requirements
Technology and data variations list	Varying I/O methods and data formats
Frequency of occurrence	Influence investigation testing timing of implementation.

2.9.5 Relationships of use case

The Three relationships of use cases are:

1. The include Relationship
2. The extend Relationship
3. The generalize Relationship

1.The include Relationship

- This is the most common and important relationship.
- It is common to have some partial behavior that is common across several use cases.
- For example, the description of paying by credit occurs in several use cases, including Process Sale, Process Rental, Contribute.
- Rather than duplicate this text, it is desirable to separate it into its own sub function use case, and indicate its inclusion.
- For example: UC1: Process Sale

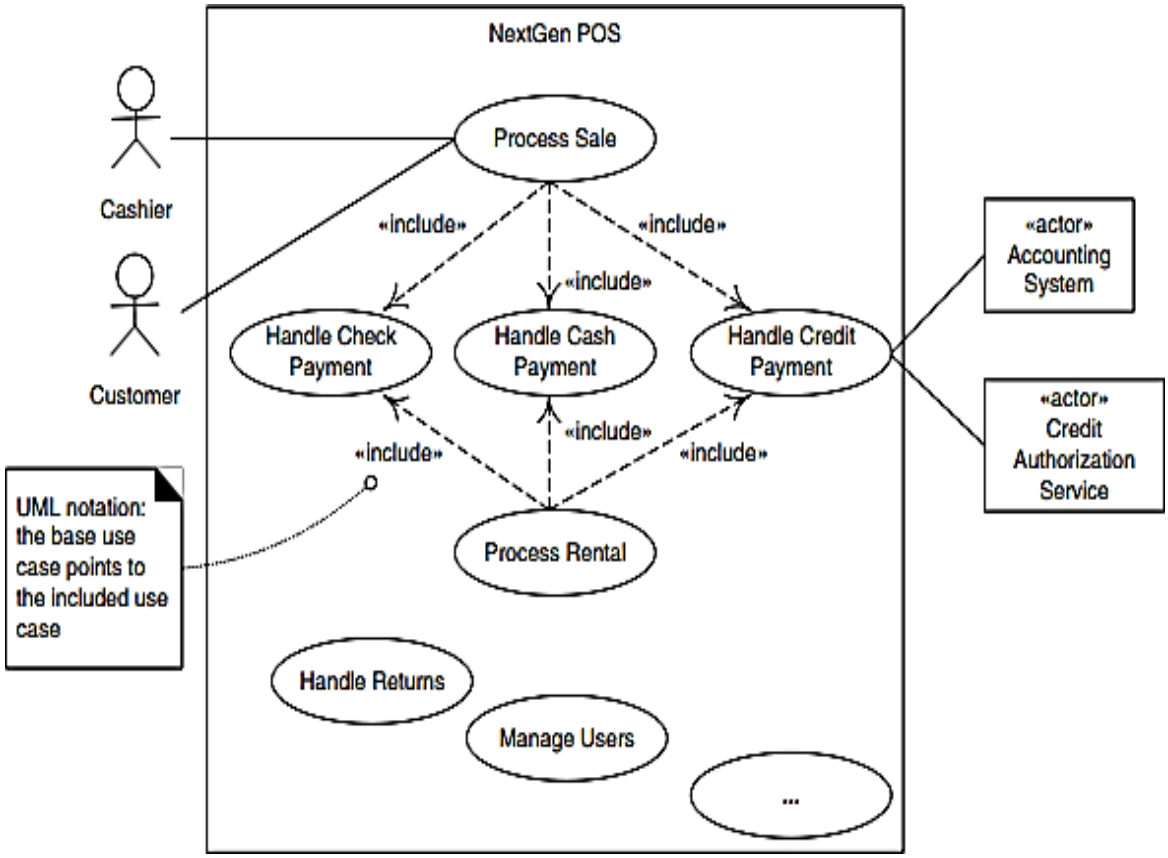


Figure 2.9: Use case include relationship in the Use-Case Model.

Main Success Scenario:

1 . Customer arrives at a POS checkout with goods and/or services to purchase. .

7. Customer pays and System handles payment.

Extensions:

7b. Paying by credit: Include Handle Credit Payment.

7c. Paying by check: Include Handle Check Payment.

UC7: Process Rental

Extensions: 6b.

Paying by credit: Include Handle Credit Payment.

UC12: Handle Credit Payment

Level: Sub function Main Success Scenario:

1. Customer enters their credit account information.

2. System sends payment authorization request to an external Payment Authorization Service System, and requests payment approval.

3. System receives payment approval and signals approval to Cashier.

4. ...

Extensions:

2a. System detects failure to collaborate with external system:

1 . System signals error to Cashier.

2. Cashier asks Customer for alternate payment.

2.9.6 The extend Relationship

The extend relationship provides an answer. The idea is to create an extending or addition use case, and within it, describe where and under what condition it extends the behavior of some base use case.

This is an example of an extend relationship. Extension points are labels in the base use case which the extending use case references as the point of extension, so that the step numbering of the base use case can change without affecting the extending use case—indirection yet again.

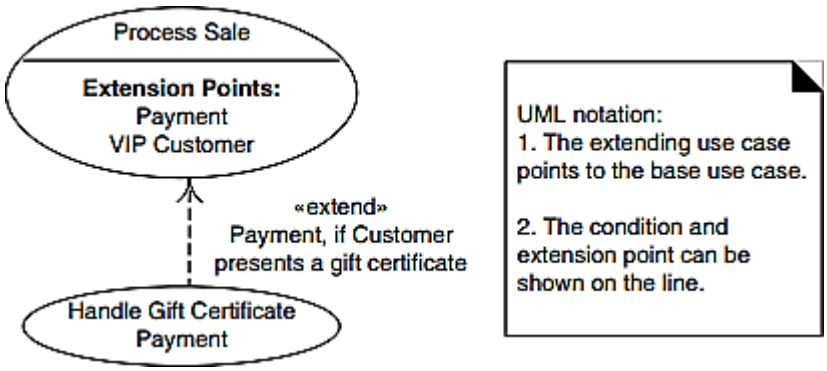


Figure 2.10:The extend relationship

2.9.7 The generalize Relationship

This is an optional relationship.

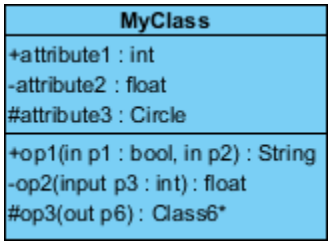
It adds another level of complexity to use cases.

2.10 Class diagram

Class diagram is a static diagram. It represents the static view of an application. Class diagram describes the attributes and operations of a class and also the constraints imposed on the system. Class diagram shows a collection of classes, interfaces, associations, collaborations, and constraints. It is also known as a structural diagram.

A class notation consists of three parts:

1. **Class Name**
    - The name of the class appears in the first partition.
2. **Class Attributes**
    - Attributes are shown in the second partition.
    - The attribute type is shown after the colon.
    - Attributes map onto member variables (data members) in code.
3. **Class Operations** (Methods)
    - Operations are shown in the third partition. They are services the class provides.
    - The return type of a method is shown after the colon at the end of the method signature.
    - The return type of method parameters are shown after the colon following the parameter name.
    - Operations map onto class methods in code



Two Perspectives of class diagram:

i) Conceptual perspective

ii)Software perspective

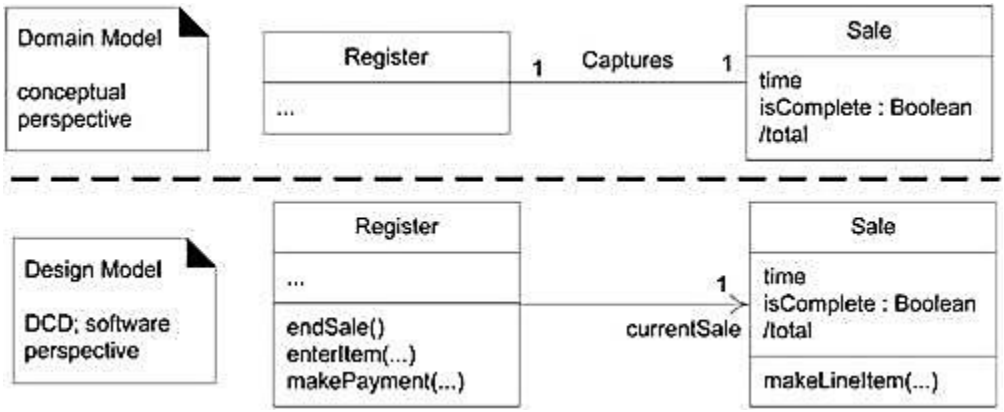


Figure 2.11: Domain model and Define model

Classifier

A UML **classifier** is "a model element that describes behavioral and structure features". In class diagrams the two most classifiers are regular classes and interfaces.

UML Attributes

They can be shown in several ways.

The full format of the attribute text notation is

visibility name : type multiplicity = default {property-string}

visibility marks include + (public), - (private), and so forth.

- a **navigability arrow** pointing from the source (*Register*) to target (*Sale*) object indicating a Register object has an attribute of one sale
- multiplicity at the target end, but not the source end
- Role name (current sale) only at the target end to show the attribute name.

Notes, Comments, Constraints, and Method Bodies

- a UML **note** or **comment** is displayed as a dog eared rectangle with a dashed line to the annotated element
- a UML **constraint**, in which case it must be encased in braces '{…}'
- a **method** body the implementation of a UML operation

Operations

The format of the operation syntax is:

visibility name (parameter-list) : return-type {property-string}

Operations are usually assumed public if no visibility is shown. The property string contains arbitrary additional information, such as exceptions that may be raised, if the operation is abstract, and so forth.

For example:

+getPlayer(name : String) : Player {exception IOException}

An operation is not a method. A UML **operation** is a declaration, with a name, parameters, return type, exceptions list, and possibly a set of *constraints* of pre-and post-conditions.

Methods - A UML **method** is the implementation of an operation; if constraints are defined, the method must satisfy them.

Keywords - A UML **keyword** is a textual adornment to categorize a model element. Most keywords are shown in guillemet (« ») but some are shown in curly braces, such as {abstract}.Some of the key words are

Keyword	Meaning	Example Usage
«actor»	classifier is an actor	in class diagram, above classifier
«interface»	classifier is an interface	in class diagram, above classifier
{abstract}	abstract element; can't be	in class diagrams, after classifier
{ordered}	a set of objects have some	in class diagrams, at an association

Stereotypes, Profiles, and Tags

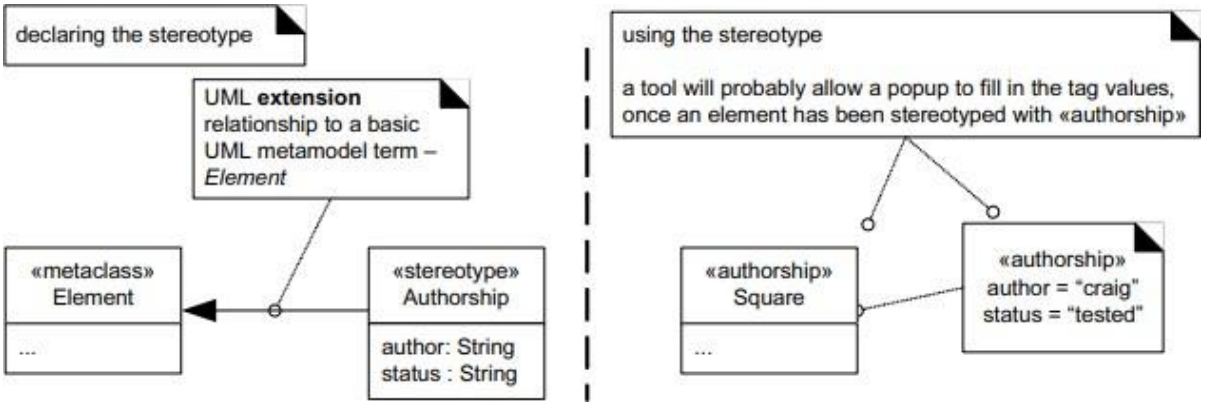


Figure 2.12: Stereotype declaration and use.

A **stereotype** represents a refinement of an existing modeling concept and is defined within a UML.

Profile is a collection of related stereotypes, tags, and constraints to specialize the use of the UML for a specific domain or platform, such as a UML profile for project management or for data modeling.

The stereotype declares a set of tags, using the attribute syntax. When an element (such as the Square class) is marked with a stereotype, all the tags apply to the element, and can be assigned values.

UML Properties and Property Strings

In the UML, a **property** is "a named value denoting a characteristic of an element. A property has semantic impact

Properties of elements may be presented in many ways, but a textual approach is to use the UML **property string** {name1=value1, name2=value2} format, such as {abstract, visibility=public}.

Generalization, Abstract Classes

Generalization in the UML is shown with a solid line and fat triangular arrow from the subclass to superclass. Generalization is a taxonomic relationship between a more general classifier and a more specific classifier. Each instance of the specific classifier is also an indirect instance of the general classifier.

Abstract classes and operations can be shown either with an *{abstract}* tag (useful when sketching UML) or by italicizing the name

Dependency

The UML includes a general **dependency relationship** that indicates that a **client** element (of any kind, including classes, packages, use cases, and so on) has knowledge of another **supplier** element and that a change in the supplier could affect the client.

Dependency is illustrated with a dashed arrow line from the client to supplier.

There are many kinds of dependency; here are some common types in terms of objects and class diagrams:

- having an attribute of the supplier type
- an attribute, a parameter variable, a local variable, a global variable, or class visibility
- receiving a parameter of the supplier type
- the supplier is a superclass or interface

Class Relationships

A class may be involved in one or more relationships with other classes.

Relationship Names

- Names of relationships are written in the middle of the association line.
- Good relation names make sense when you read them out loud:
  - "Every spreadsheet **contains** some number of cells",
  - "an expression **evaluates to** a value"
- They often have a **small arrowhead to show the direction** in which direction to read the relationship, e.g., expressions evaluate to values, but values do not evaluate to expressions.

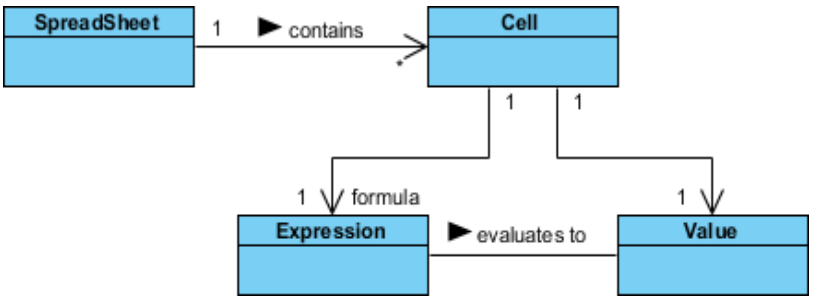
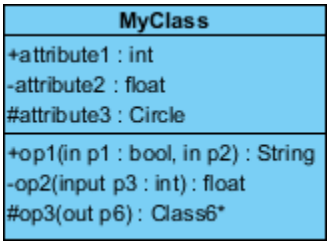


Figure 2.13 : Class Visibility Examples



In the example above:

- attribute1 and op1 of MyClassName are public
- attribute3 and op3 are protected.
- attribute2 and op2 are private.

Multiplicity

How many objects of each class take part in the relationships and multiplicity can be expressed as:

- Exactly one - 1
- Zero or one - 0..1
- Many - 0..* or *
- One or more - 1..*
- Exact Number - e.g. 3..4 or 6
- Or a complex relationship - e.g. 0..1, 3..4, 6.* would mean any number of objects other than 2 or 5

2.11 UML Interaction Diagrams

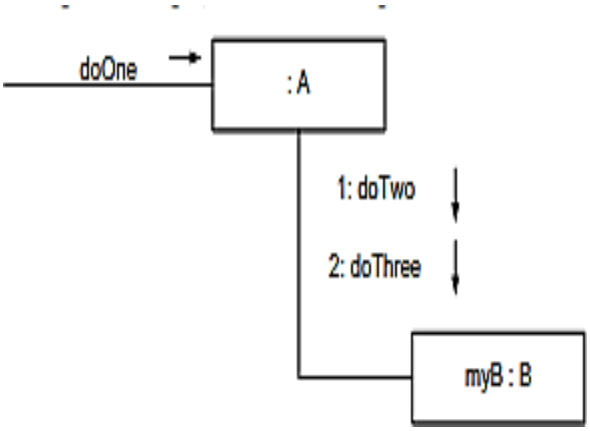
An interaction diagram is a type of UML diagram that is used to capture the interactive behavior of a system. Interaction diagrams focus on describing the flow of messages within a system, providing context for one or more lifelines within a system. In addition, interaction diagrams can be used to represent the ordered sequences within a system and act as a means of visualizing real-time data via UML. There are two common types:

1. Sequence diagrams.
2. communication diagrams.

2.11.1 Communication diagrams

A communication diagram is also called as collaboration diagram, extension of object diagram that shows the objects along with the messages that travel from one to another. In addition to the

associations among objects, communication diagram shows the messages the objects send each other.



Communication diagram symbols and elements Symbols

Numbers: Numbers show the order in which messages are sent, as well as how many messages are required to complete a process.

Rectangles: A simple shape with a simple purpose, rectangles are used to show the objects in the system. They are labeled with the object's name.

Lines: Lines connecting rectangles show the relationships between two or more objects

Arrows:These show the flow of messages sent and received throughout the process in the diagram.

Elements: Elements are the components of a system, represented in diagram form.

Objects: There are two types of objects presented in a communication diagram.

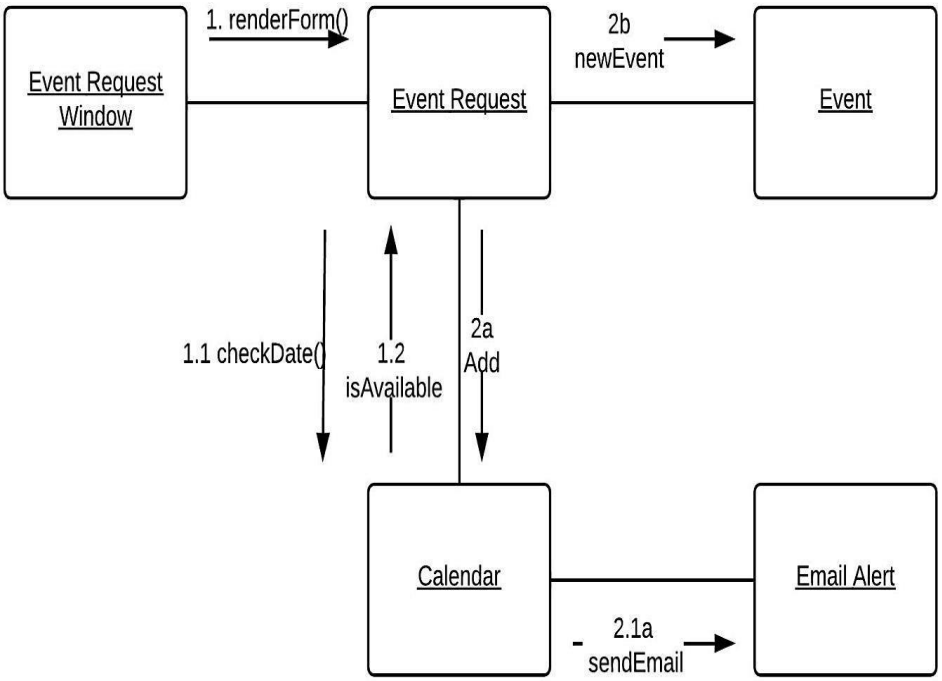
i. Supplier objects – These objects supply the method, or process. Supplier objects are the message receivers.

ii. Client objects – These objects send messages to request supplier methods.

Links:Lines represent the links between objects. They show which objects are connected, and how. They also show which objects send and receive messages. Sometimes, an object sends messages to itself, which is represented with a loop.

Messages:While lines represent links, arrows represent the flow of messages. Client and supplier objects are connected with arrows, and labeled with numbers. Messages are numbered in sequential order; 1, 2, 3, for example. The numbers on a diagram are also for the reader's benefit, because you know which messages to read first. If a message is attached to a parent message, it

can be expressed with multiple numbers and decimal points. There will also usually be a short description next to each number.



There are two types of messages:

i. Synchronous messages – After a client object sends a message, it must wait for a response from the supplier. These messages are represented by a solid line, and a filled in, triangular arrowhead.

ii. Asynchronous messages – The client object, after sending a message, does not need to wait for a response before continuing. These messages are shown with a dashed line and a 'hollow' arrowhead.

In the example below, the communication diagram explains the process to add an event to a calendar. Even in simple examples like this one, you'll notice the exact commands and requests being shared between various steps in the process.

The numbers on each line represent the order and options in which they are activated. We know that some actions happen concurrently because of the use of letters.

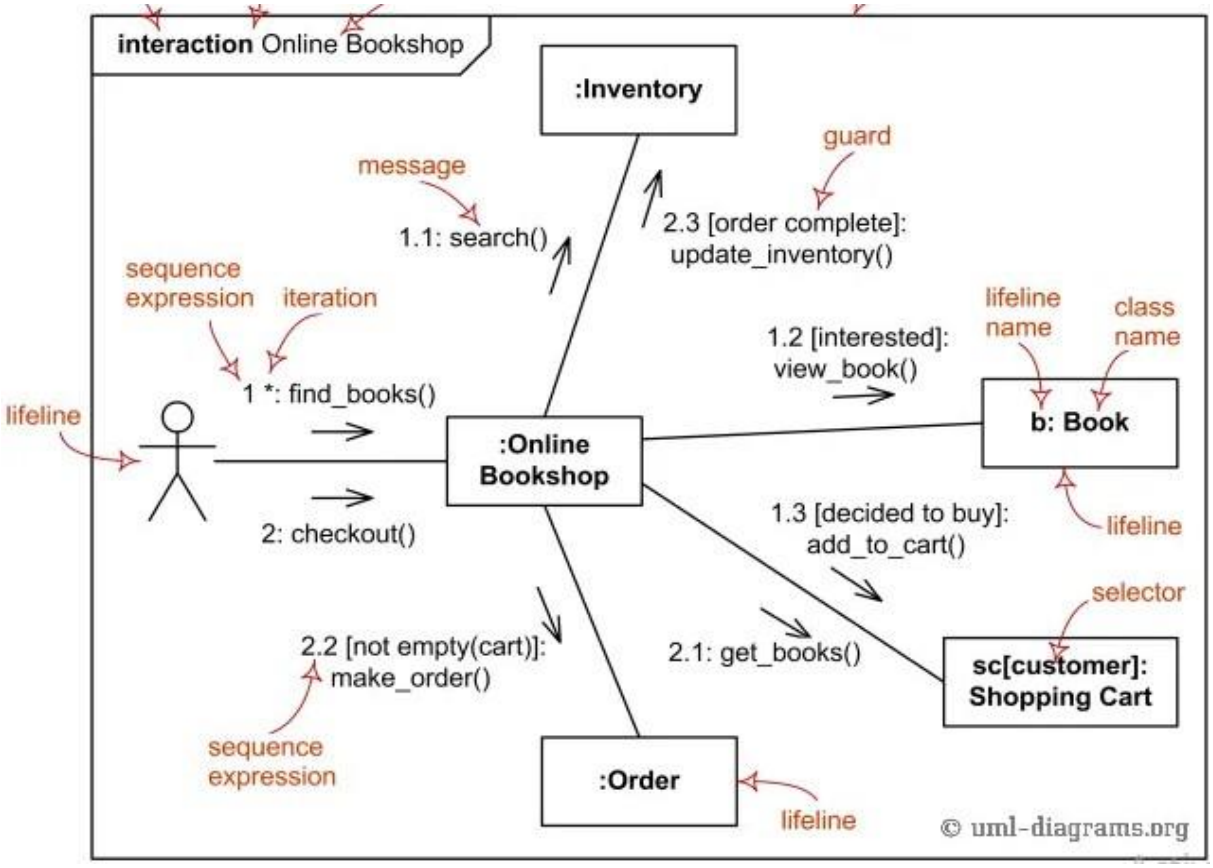


Figure 2.14: Communication diagram for online Book shopping

2.11.2 Sequence diagrams

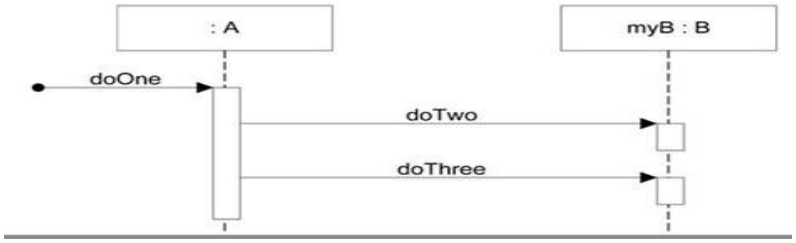


Figure 2.15: Sequence diagram

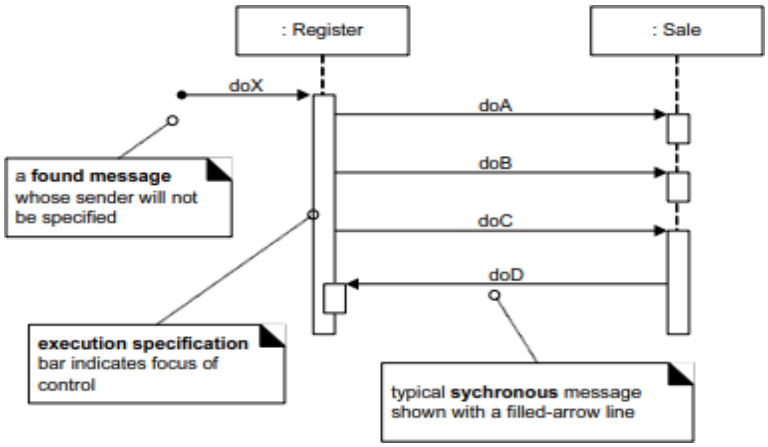
Sequence diagrams illustrate interactions in a kind of fence format, in which each new object is added to the right. A **sequence diagram** shows object interactions arranged in time sequence. It depicts the objects and classes involved in the scenario and the sequence of messages exchanged between the objects needed to carry out the functionality of the scenario.

Basic Sequence Diagram Notation

Messages

Each message between objects is represented with a message expression on a filled-arrowed, solid line between the vertical lifelines. The time ordering is organized from top to bottom of lifelines.

Focus of Control and Execution Specification Bars



Sequence diagrams may also show the focus of control using an execution specification bar. The bar is optional.

Reply or Returns

There are two ways to show the return result from a message:

- Using the message syntax returnVar = message(parameter).
- Using a reply (or return) message line at the end of an activation bar.

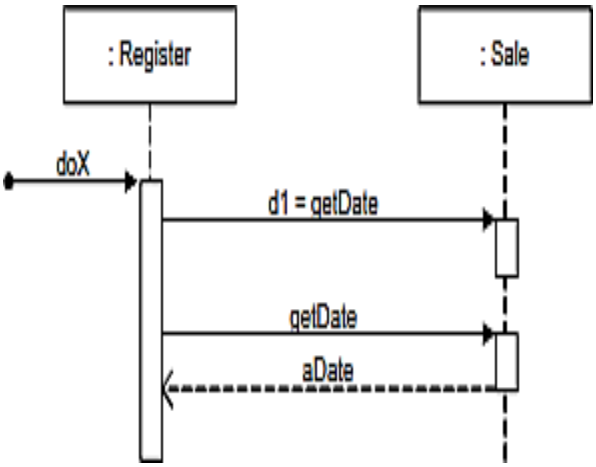


Figure 2.16 : Two ways to show a return result from a message

Messages to "self" or "this"

Message being sent from an object to itself by using a nested activation bar.

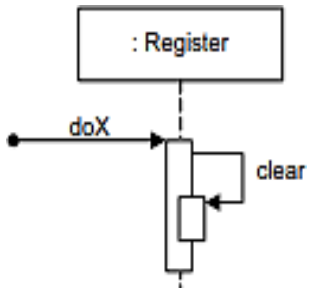


Figure 2.17: Messages to "this."

Creation of Instance

ObjA creates an objects of another class,this object is named ObjB.Object creation notation is shown in dashed line with filled arrow (synchronous message), or open (stick arrow) if an asynchronous call.

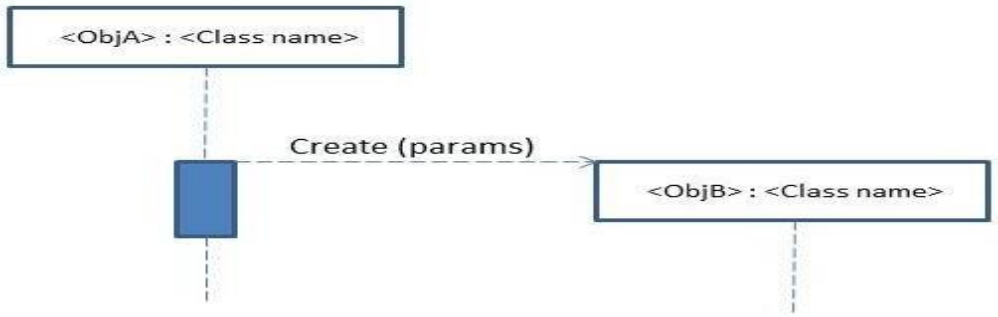


Figure 2.18: Instance creation and object lifelines.

Object Destruction

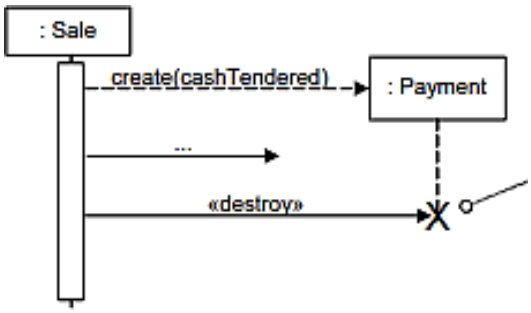


Figure 2.19: Object destruction.

The explicit destruction of an object can be shown with <<destroy>>sterotype message with larger X.

Frames

Frames are regions or fragments of the diagrams; they have an operator or label (such as loop) and a guard .The [boolean test] guard should be placed over the lifeline to which it belongs.

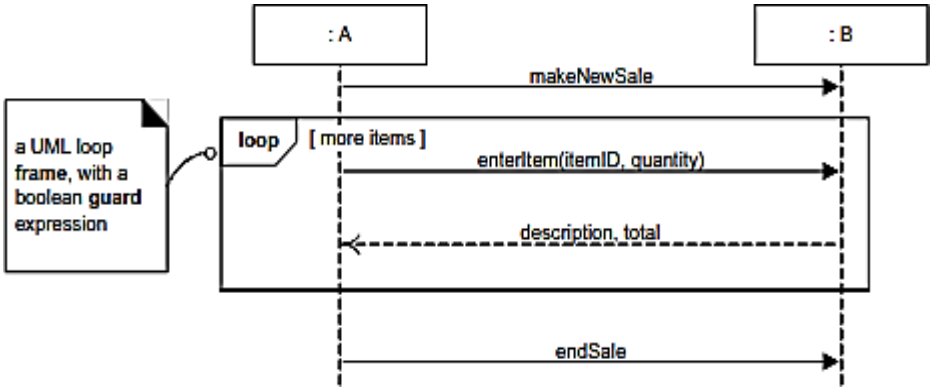
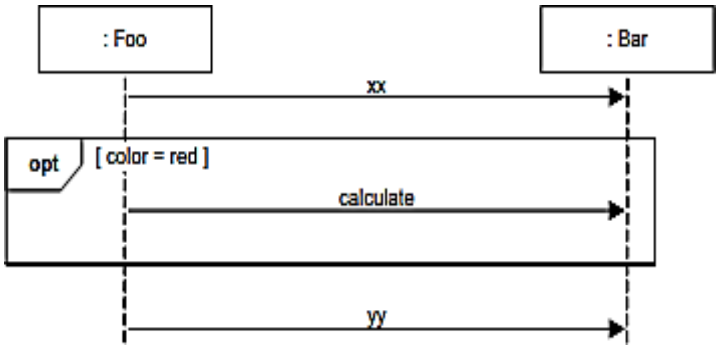


Figure 2.20: Example UML frame.

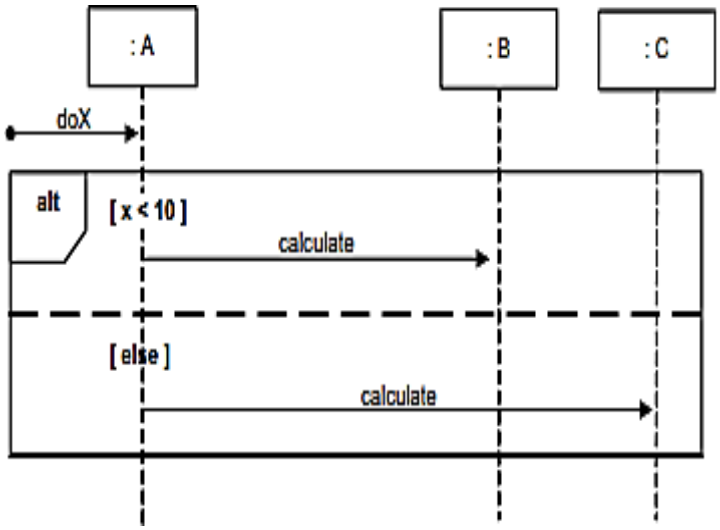
Conditional Messages

An OPT frame is placed around one or more messages. Notice that the guard is placed over the related lifeline.



Mutually Exclusive Conditional Messages

An ALT frame is placed around the mutually exclusive alternatives

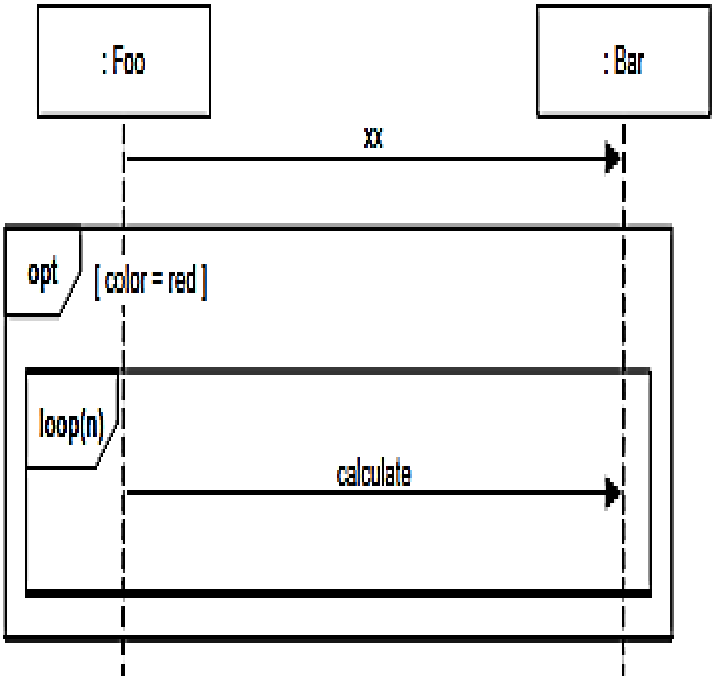


Iteration Over a Collection

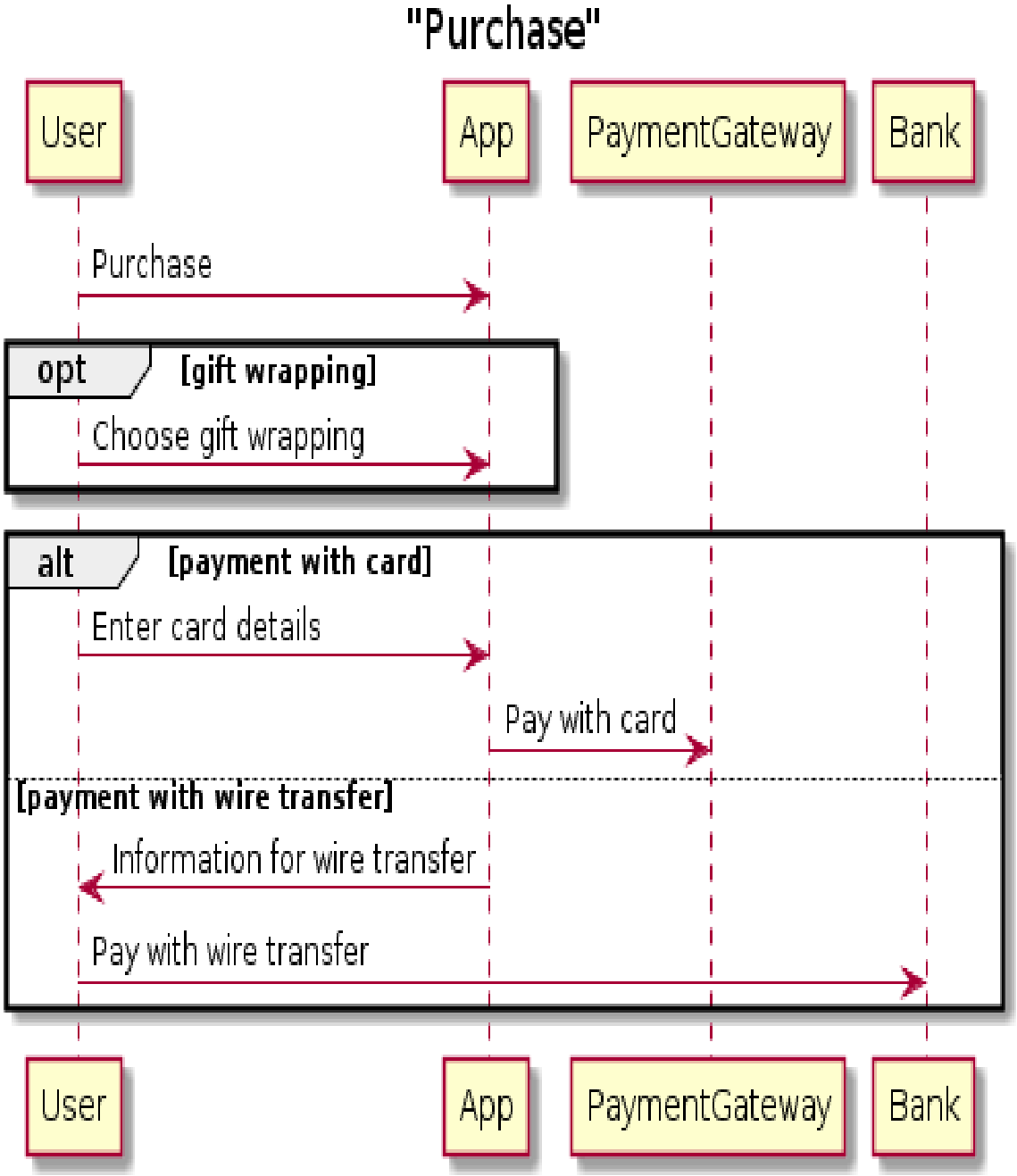
A common algorithm is to iterate over all members of a collection (such as a list or map), sending the same message to each.

Nesting of Frames

Frames can be nested.

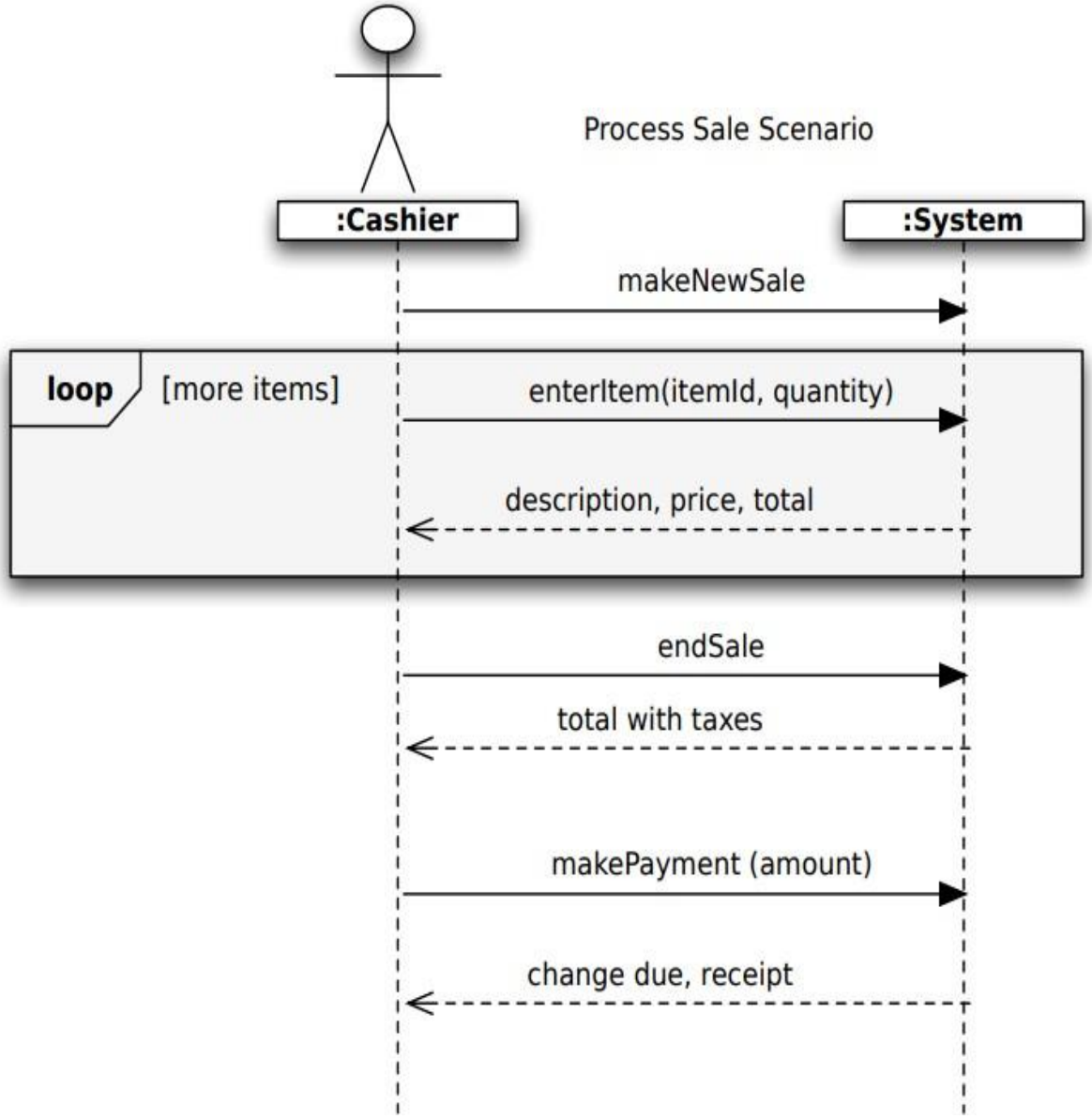


Example:



System Sequence Diagrams for process sale scenario

A system sequence diagram (SSD) is a fast and easily created artifact that illustrates input and output events related to the systems under discussion. They are input to operation contracts and most importantly object design.



2.12 Activity Diagrams

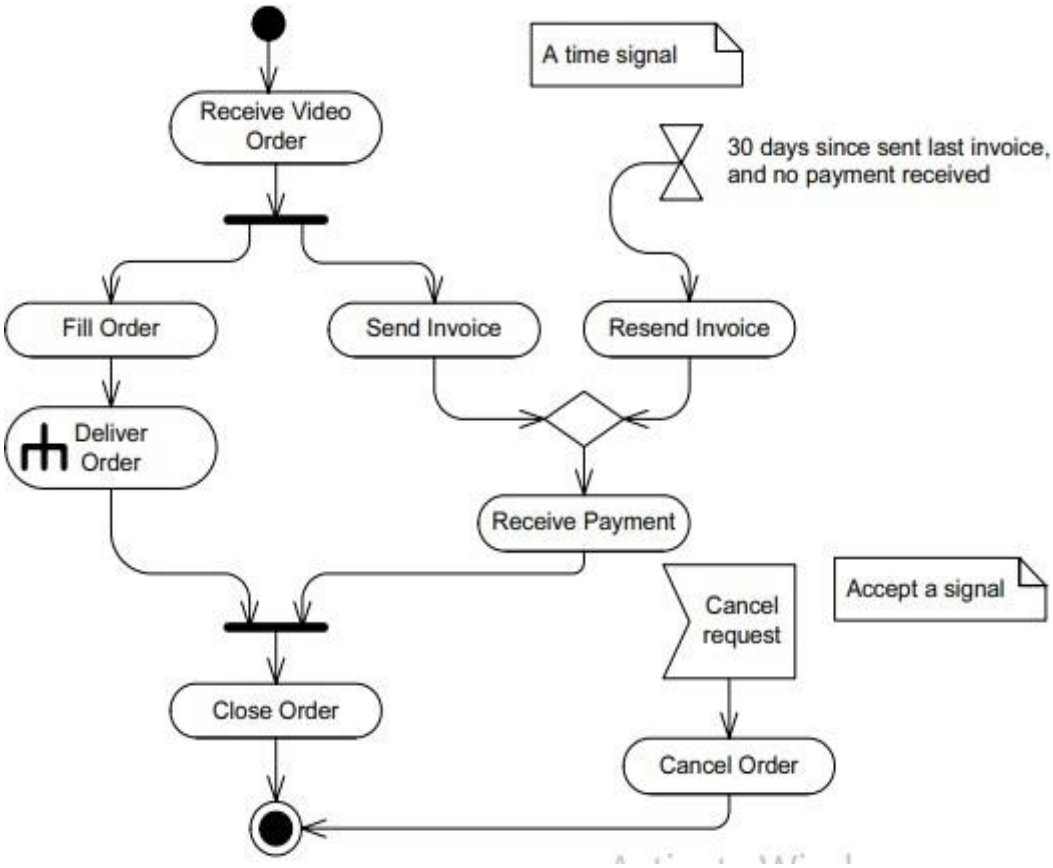
Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams are intended to model both computational and organizational processes (i.e., workflows), as well as the data flows intersecting with the related activities. Although activity diagrams primarily show the overall flow of control.

2.12.1 Basic UML diagram notations

Symbol	Name	Description
	Start symbol	Represents the beginning of a process or workflow in an activity diagram.
Activity	Activity symbol	Indicates the activities that make up a modeled process. These symbols are the main building blocks of an activity diagram.
	Connector symbol	Shows the directional flow, or control flow, of the activity.
	Joint symbol/ Synchronization bar	Combines two concurrent activities. Represented with a thick vertical or horizontal line.
	Fork symbol	Splits a single activity flow into two concurrent activities.
	Decision symbol	Represents a decision and always has at least two paths branching out with condition text to allow users to view options.
	Note symbol	Leave notes for added clarity and specification.
	Send signal symbol	Indicates that a signal is being sent to a receiving activity.
	Receive signal symbol	Demonstrates the acceptance of an event.
	Flow final symbol	Represents the end of a specific process flow.This symbol shouldn't represent the end of all flows in an activity.
[condition]	Condition text	Placed next to a decision marker to let you know under what condition an activity flow should split off in that direction.
	End symbol	Marks the end state of an activity.

2.12.2 Signals

Signals are useful when you need to model events such as time triggering an action or a cancellation request.



2.12.3 Rake symbols

Rake symbol indicates an activity is expanded in another activity diagram also it is called as action symbol indicates that the activity being invoked is described on another diagram. For example, to transform light into video signals, the Operate Camera activity invokes other activities that perform various subtasks using call behavior actions.

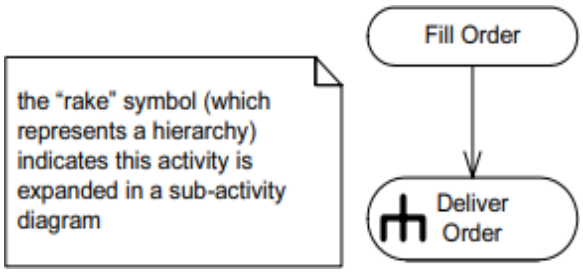


Figure2.21 :An activity will be expanded in another diagram.

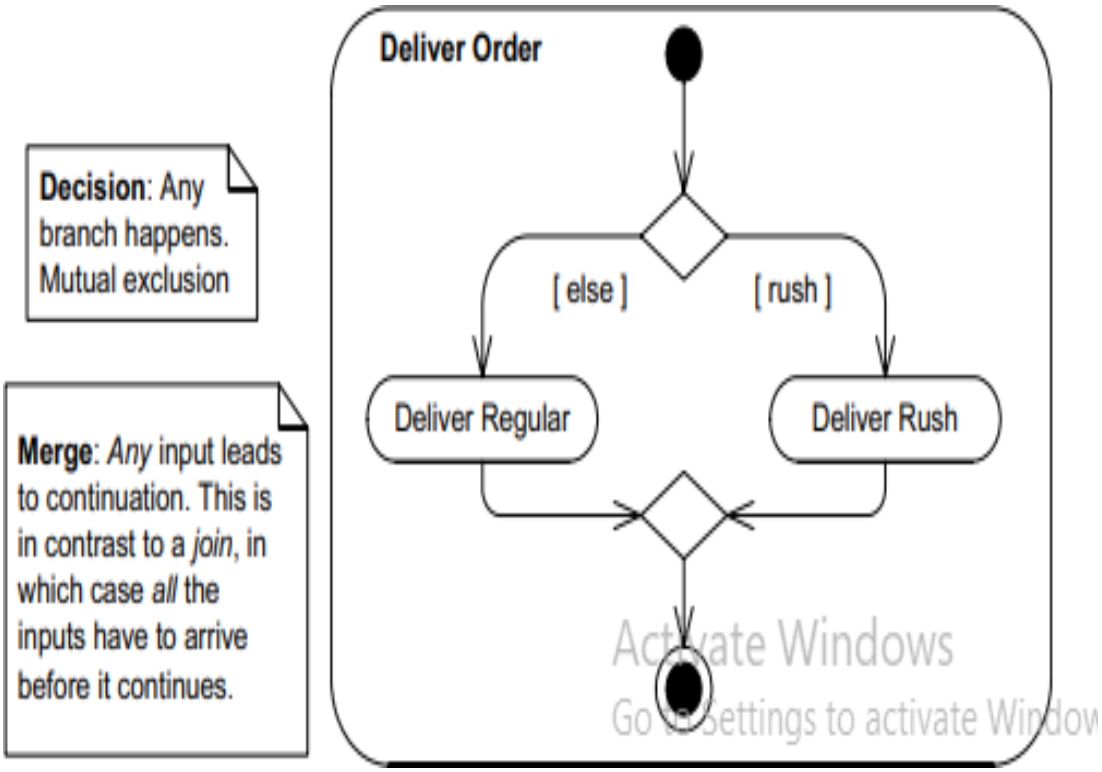


Figure 2.22: The expansion of an activity.

Guidelines

- This technique proves most valuable for very complex processes, usually involving many parties.
- If modeling a business process, take advantage of the "rake" notation and sub-activity diagrams.
- Strive to make the level of abstraction of action nodes roughly equal within a diagram.

Activity Diagrams in the UP

- Activity diagrams are especially applicable within the business Modelling discipline of the UP.
- The purpose of Business Modeling is to understand and communicate "the structure and the dynamics of the organization in which a system is to be deployed".
- A key artifact of the Business Modeling discipline is the Business Object Model .

- Business Object Model visualizes how a business works, using UML class, sequence, and activity diagrams.
- Thus, activity diagrams are especially applicable within the Business Modeling discipline of the UP.

- **<u>Example: NextGen Activity Diagram(Case Study)</u>**

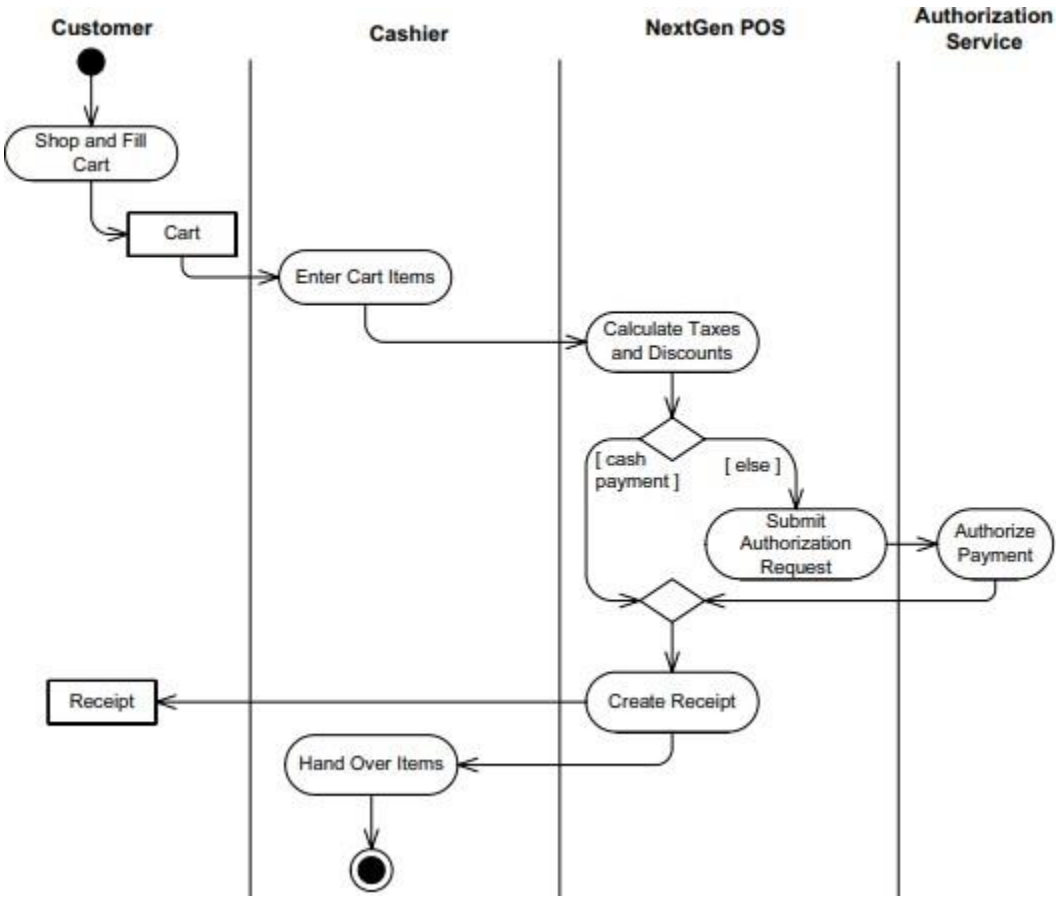


Figure2.23: Modeling the Process Sale use case with an activity diagram.

2.13 UML State Machine Diagrams

A UML state machine diagram, illustrates the interesting events and states of an object, and the behavior of an object in reaction to an event. Transitions are shown as arrows, labeled with their event. States are shown in rounded rectangles. It is common to include an initial pseudo-state, which automatically transitions to another state when the instance is created.

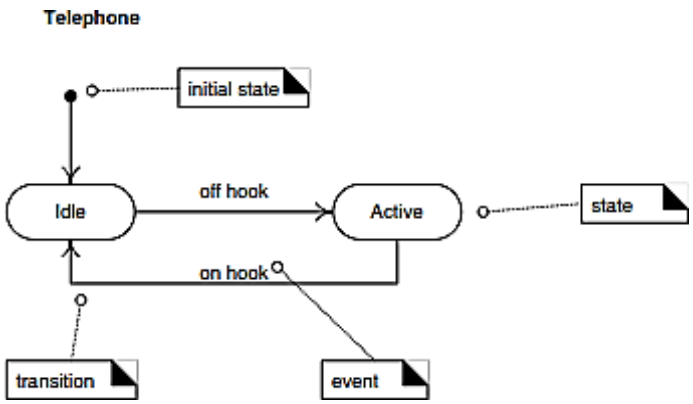
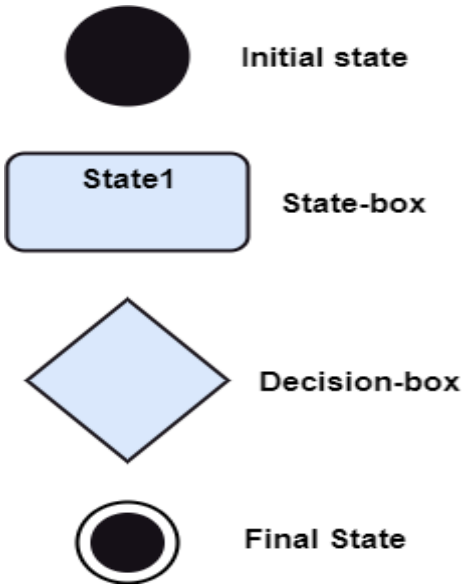


Figure 2.24:State machine diagram for a telephone

A state machine diagram shows the lifecycle of an object: what events it experiences, its transitions, and the states it is in between these events.

2.13.1 Notation of a State Machine Diagram

Following are the notations of a state machine diagram enlisted below:



i. Initial state: It defines the initial state (beginning) of a system, and it is represented by a black filled circle.

ii. Final state: It represents the final state (end) of a system. It is denoted by a filled circle present within a circle.

iii. Decision box: It is of diamond shape that represents the decisions to be made on the basis of an evaluated guard.

iv. Transition: A change of control from one state to another due to the occurrence of some event is termed as a transition. It is represented by an arrow labeled with an event due to which the change has ensued.

v. State box: It depicts the conditions or circumstances of a particular object of a class at a specific point of time. A rectangle with round corners is used to represent the state box.

2.13.2 Events, States, and Transitions

i. An event is a significant or noteworthy occurrence,For example A telephone receiver is taken off the hook.

ii. A state is the condition of an object at a moment in time the time between events.

For exampleA telephone is in the state of being "idle" after the receiver is placed on the hook and until it is taken off the hook

Unified Modeling Language defines three types of states:

- **Simple state**-They do not have any substate.
- **Composite state-** It consists of nested states (substates), such that it does not contain more than one initial state and one final state. It can be nested to any level.
- **Submachine state-** he submachine state is semantically identical to the composite state, but it can be reused.

iii. A transition is a relationship between two states that indicates that when an event occurs, the object moves from the prior state to the subsequent state. For exampleWhen the event "off hook" occurs, transition the telephone from the "idle" to "active" state.

2.13.3 State-Independent and State-Dependent Objects

State-independent: If an object always responds the same way to an event, then it is considered state-independent (or modeless) with respect to that event. For example, if an object receives a message, and the responding method always does the same thing. The object is state-independent with respect to that message.

State dependent: If, for all events of interest, an object always reacts differently to events depending on their state or mode is called state dependent objects.

2.13.4 Complex Reactive Objects

1. Physical Devices controlled by software

   Phone, car, microwave oven: They have complex and rich reactions to events, and the reaction depends upon their current mode.

2. Transactions and related Business Objects

   How does a business object (a sale, order, payment) react to an event? For example, what should happen to an Order if a cancel event occurs? And understanding all the events and states that a Package can go through in the shipping business can help with design, validation, and process improvement.

3. Role MutatorsThese are objects that change their role.

   A Person changing roles from being a civilian to a veteran. Each role is represented by a state.

2.13.5 Protocols and Legal Sequences

a) Communication Protocols TCP, and new protocols, can be easily and clearly understood with a state machine diagram. The diagram illustrates when operations are legal. For example, a TCP "close" request should be ignored if the protocol handler is already in the "closed" state.

b) **UI Page/Window Flow or Navigation** When doing UI modeling, it can be useful to understand the legal sequence between Web pages or windows; this is often complex. A state machine is a great tool to model UI navigation

c) **UI Flow Controllers or Sessions** This is related to UI navigation modeling, but specifically focused on the server-side object that controls page flow. These are usually server-side objects representing an ongoing session or conversations with a client. For example, a Web application that remembers the state of the session with a Web client and controls the transitions to new Web pages, or the modified display of the current Web page, based upon the state of the session and the next operation that is received.

d) Use Case System Operations system operations for Process Sale: makeNewSale, enterItem, should arrive in a legal order; for example, endSale should only come after one or more enterItem operations. Usually, the order is trivially obvious, but if complex, a state machine can model this, treating the use case itself as an object.

e) **Individual UI Window Event Handling** Understanding the events and legal sequences for one window or form. For example, the Edit-Paste action is only valid if there is something in the "clipboard" to paste.

2.13.6 Transition Actions and Guards - A transition can cause an action to fire. In a software implementation, this may represent the invocation of a method of the class of the state machine diagram.A transition may also have a conditional guard or boolean test. The transition only occurs if the test passes.

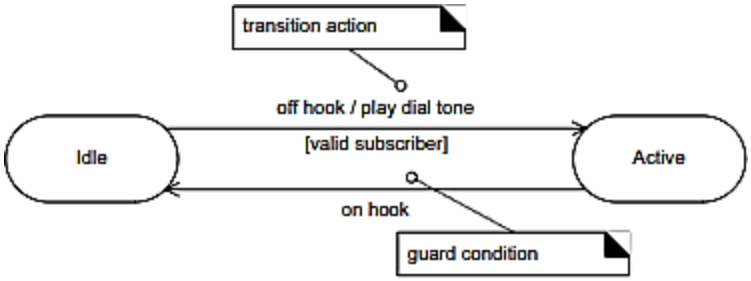


Figure 2.25:Transition action and guard notation.

2.13.7 Nested States

A state allows nesting to contain sub states; a sub state inherits the transitions of its super state (the enclosing state). Sub states may be graphically shown by nesting them in a super state box.

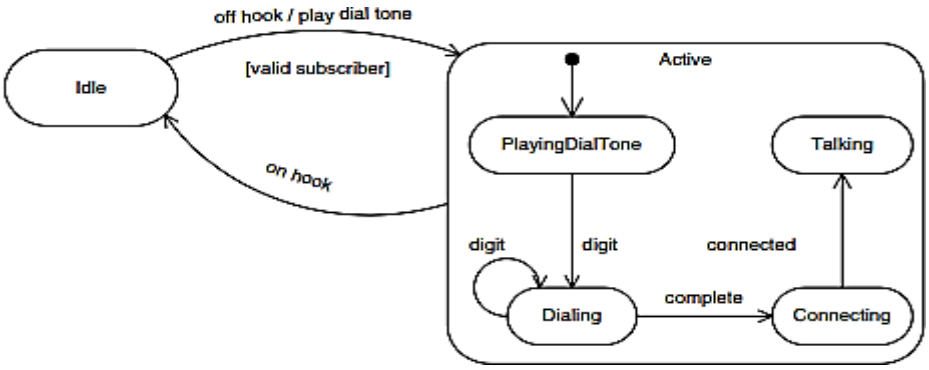


Figure 2.26:Nested states.

For example, when a transition to the Active state occurs, creation and transition into the PlayingDialTone substate occurs.

2.13.7 Example: NextGen Use Case State Machine Diagram

There are no really interesting complex reactive objects in the case studies, a state machine diagram to show legal sequencing of use case operation.

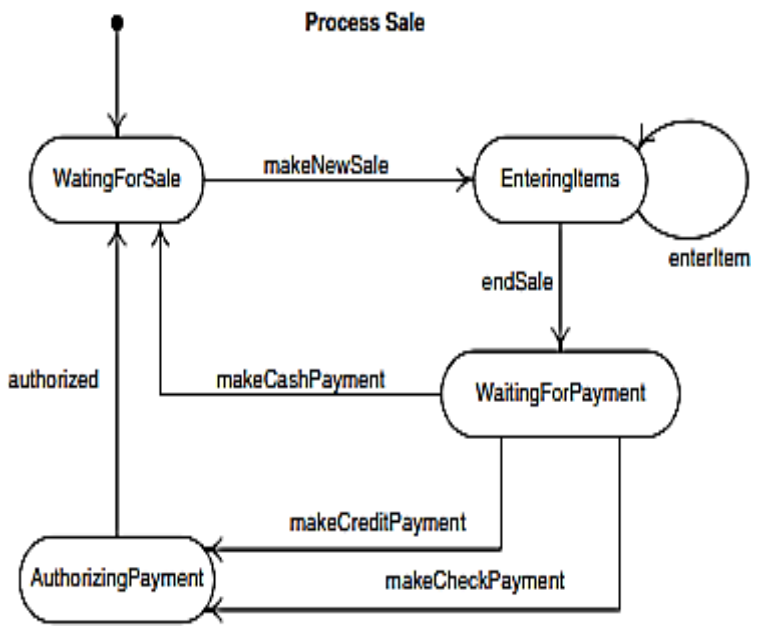


Figure 2.27: sample state machine for legal sequence of use case operations.

2.14 Functional Modeling

Functional Modelling gives the process perspective of the object-oriented analysis model and an overview of what the system is supposed to do. It defines the function of the internal processes in the system with the aid of Data Flow Diagrams (DFDs). It depicts the functional derivation of the data values without indicating how they are derived when they are computed, or why they need to be computed.

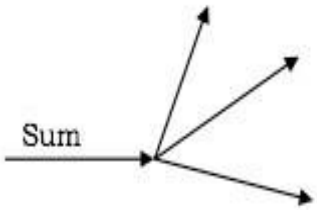
2.14.1 Data Flow diagram

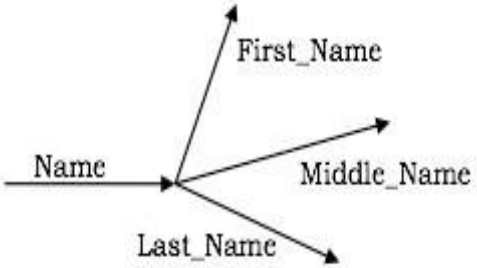
Function modelling is represented with the help of DFDs. DFD is the graphically representation of data. It shows the input, output and processing of the system .When we are trying to create our own business, website, system, project then there is need to find out how information passes from one process to another so all are done by DFD. There are number of levels in DFD but upto third level DFD is sufficient for understanding of any system.

The basic components of the DFD are:

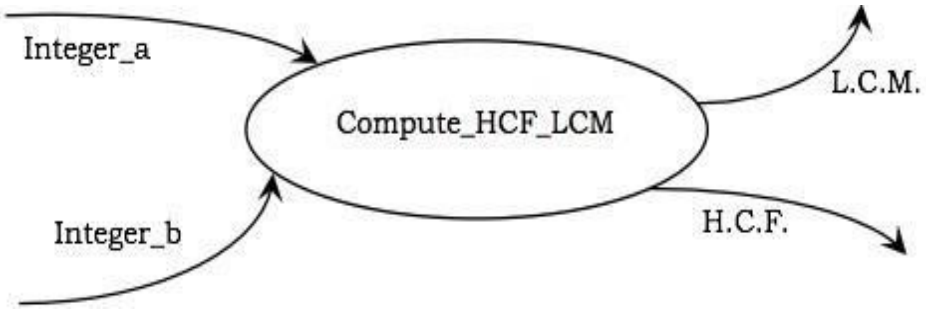
1. **External Entity :** External entity is the entity that takes information and gives information to the system. It is represented with rectangle.

2. Data Flow : The data passes from one place to another is shown by data flow. A data flow is represented by a directed arc or an arrow, labelled with the name of the data item that it carries.

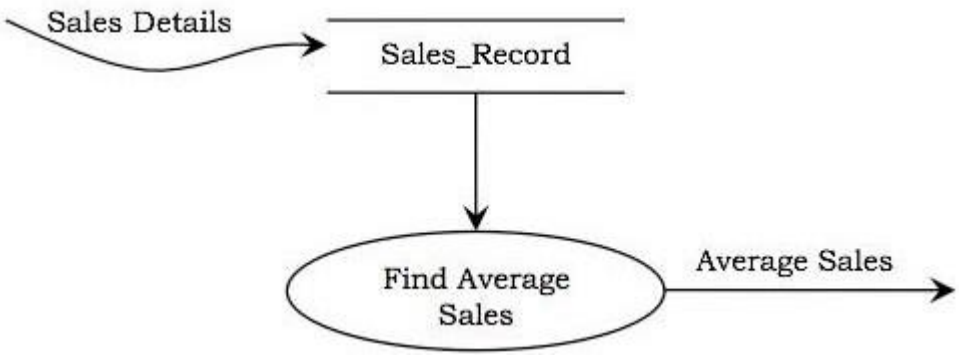




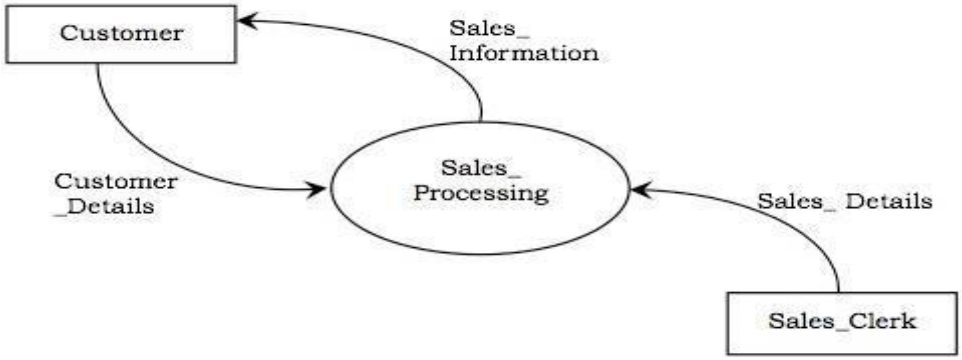
3. Process : It is also called function symbol .It is used to process all the information .If there are calculations so all are done in the process part . A process is represented as an ellipse with its name written inside it and contains a fixed number of input and output data values.The following figure shows a process Compute_ HCF_LCM that accepts two integers as inputs and outputs their HCF (highest common factor) and LCM (least common multiple).



4. Data Store : It is used to store the information and retrieve the stored information . A data store is represented by two parallel lines containing the name of the data store. Each data store is connected to at least one process. Input arrows contain information to modify the contents of the data store, while output arrows contain information retrieved from the data store. When a part of the information is to be retrieved, the output arrow is labelled. An unlabelled arrow denotes full data retrieval. A two-way arrow implies both retrieval and update.



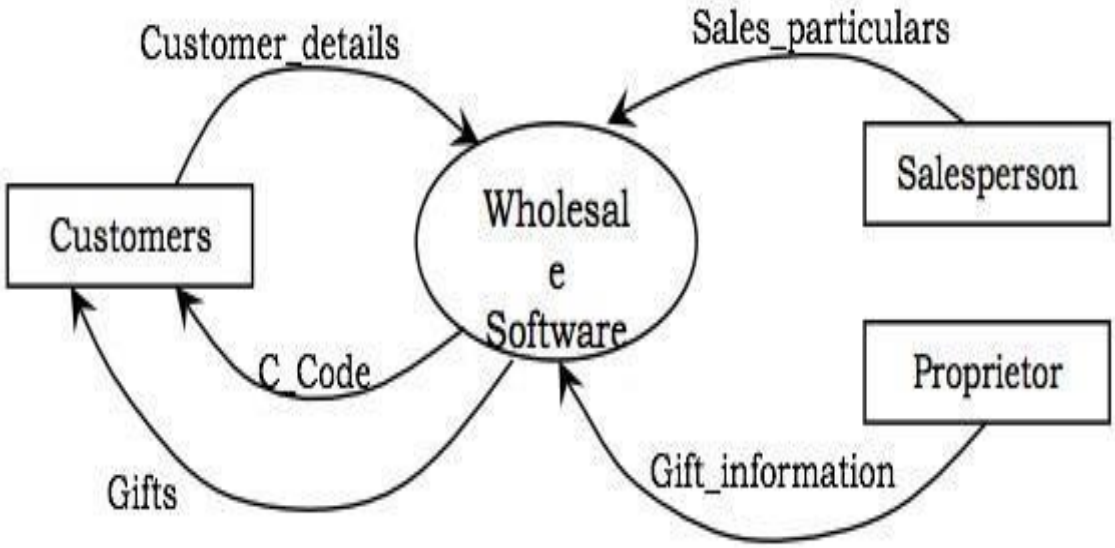
5. **Actors:** Actors are the active objects that interact with the system by either producing data and inputting them to the system, or consuming data produced by the system. In other words, actors serve as the sources and the sinks of data. An actor is represented by a rectangle. Actors are connected to the inputs and outputs and lie on the boundary of the DFD.



Example − Let us consider a software system, Wholesaler Software, that automates the transactions of a wholesale shop. The shop sells in bulks and has a clientele comprising of merchants and retail shop owners. Each customer is asked to register with his/her particulars and is given a unique customer code, C_Code. Once a sale is done, the shop registers its details and sends the goods for dispatch. Each year, the shop distributes Christmas gifts to its customers, which comprise of a silver coin or a gold coin depending upon the total sales and the decision of the proprietor.The functional model for the Wholesale Software is given below. The figure below shows the top-level DFD. It shows the software as a single process and the actors that interact with it.

The actors in the system are −

- Customers
- Salesperson
- Proprietor



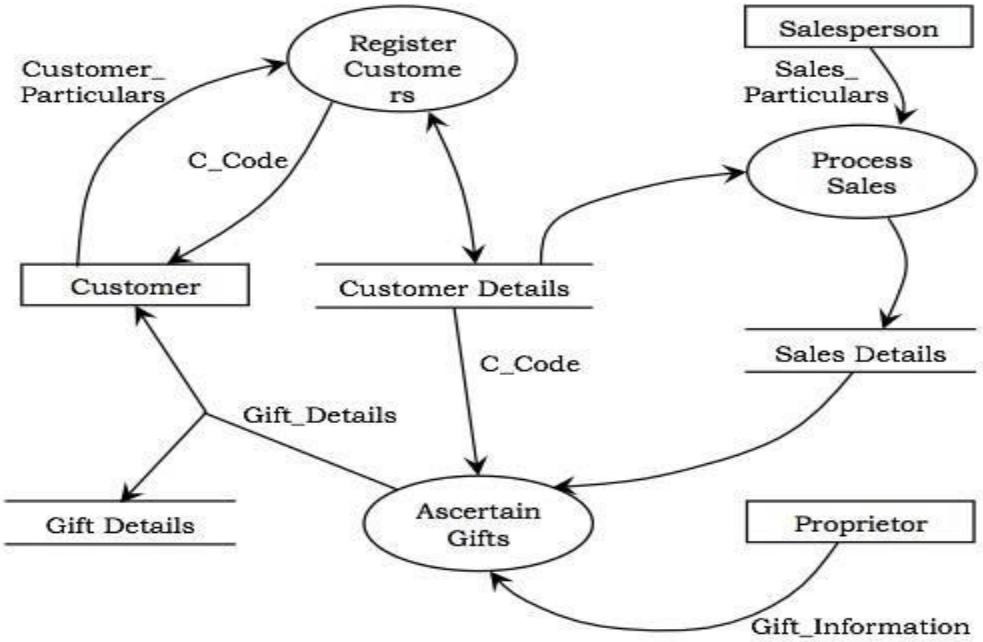
In the next level DFD, as shown in the following figure, the major processes of the system are identified, the data stores are defined and the interaction of the processes with the actors, and the data stores are established.

In the system, three processes can be identified, which are −

- Register Customers
- Process Sales
- Ascertain Gifts

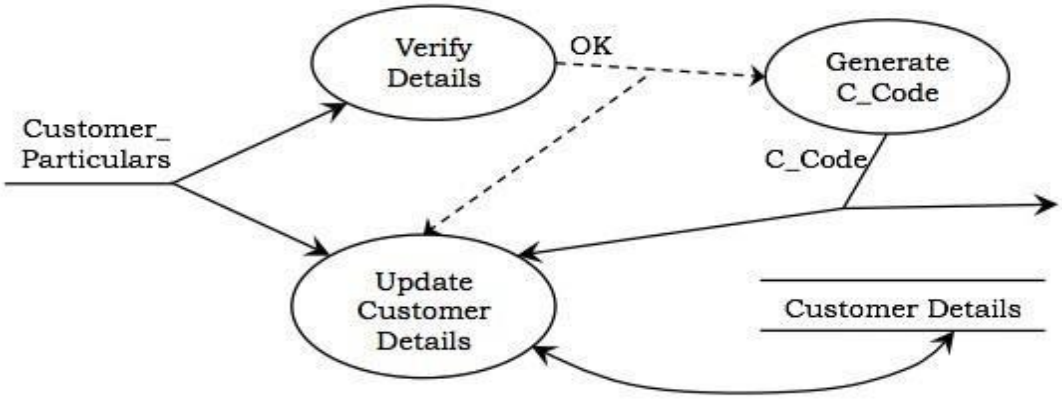
The data stores that will be required are −

- Customer Details
- Sales Details
- Gift Details



DFD of Wholesale Software

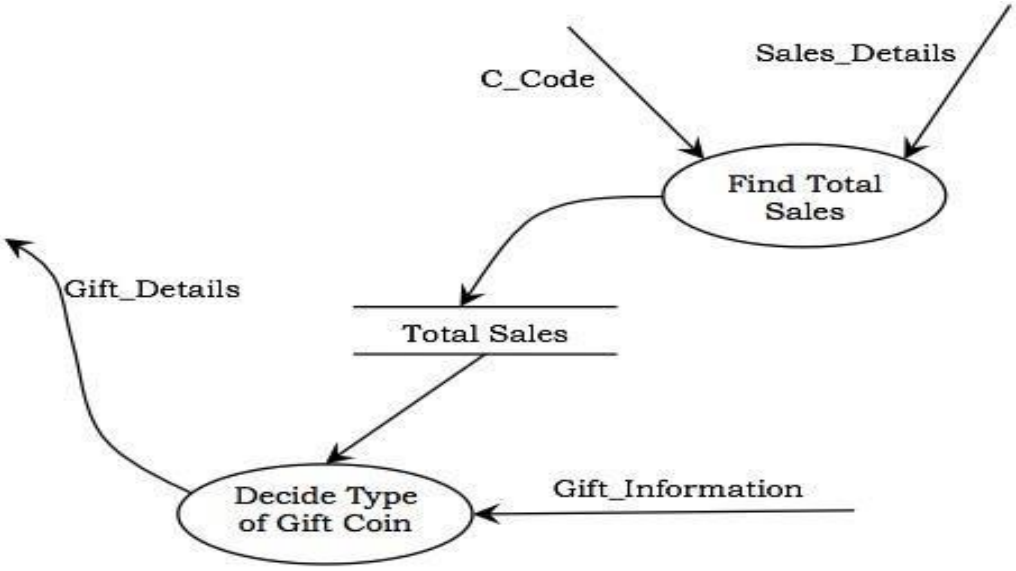
The following figure shows the details of the process Register Customer. There are three processes in it, Verify Details, Generate C_Code, and Update Customer Details. When the details of the customer are entered, they are verified. If the data is correct, C_Code is generated and the data store Customer Details is updated.



DFD of Customer Process

The following figure shows the expansion of the process Ascertain Gifts. It has two processes in it, Find Total Sales and Decide Type of Gift Coin. The Find Total Sales process computes the yearly total sales corresponding to each customer and records the data. Taking this record and the

decision of the proprietor as inputs, the gift coins are allotted through Decide Type of Gift Coin process.



DFD of Gift Process

Part A (Two Marks)

1. What is requirement engineering?

Requirement engineering is the process of establishing the services that the customer requires from the system and the constraints under which it operates and is developed.

2. What are the various types of traceability in software engineering?

i. Source traceability – These are basically the links from requirement to stakeholders

ii. Requirements traceability – These are links between dependent requirements.

iii. Design traceability – These are links from requirements to design.

3. Define software prototyping.

Software prototyping is defined as a rapid software development for validating the requirements.

4. What are the benefits of prototyping?

i. Prototype serves as a basis for deriving system specification.

ii. Design quality can be improved.

iii. System can be maintained easily.

iv. Development efforts may get reduced.

v. System usability can be improved.

5. What are the prototyping approaches in software process?

i. Evolutionary prototyping – the initial prototype is prepared and it is then refined through number of stages to final stage.

ii. Throw-away prototyping – a rough practical implementation of the system is produced.

The requirement problems can be identified from this implementation.

6. What are the advantages of evolutionary prototyping?

i. Fast delivery of the working system.

ii. User is involved while developing the system.

iii. More useful system can be delivered.

iv. Specification, design and implementation work in co-ordinate manner.

7. What are the various Rapid prototyping techniques?

i. Dynamic high level language development.

ii. Database programming.

iii. Component and application assembly.

8. What is the use of User Interface prototyping?

This prototyping is used to pre-specify the look and feel of user interface in an effective way.

9. What are the characteristics of SRS?

i. Correct – The SRS should be made up to date when appropriate requirements are identified.

ii. Unambiguous – When the requirements are correctly understood then only it is possible to write an unambiguous software.

iii. Complete – To make SRS complete, it should be specified what a software designer wants to create software.

iv. Consistent – It should be consistent with reference to the functionalities identified.

v. Specific – The requirements should be mentioned specifically.

vi. Traceable – What is the need for mentioned requirement?

10. What is data modeling?

Data modeling is the basic step in the analysis modeling. In data modeling the data objects are examined independently of processing. The data model represents how data are related with one

another.

11. What is a data object?

Data object is a collection of attributes that act as an aspect, characteristic, quality, or descriptor of the object.

12. What are attributes?

Attributes are the one, which defines the properties of data object.

13. What is cardinality in data modeling?

Cardinality in data modeling, cardinality specifies how the number of occurrences of one object is related to the number of occurrences of another object.

14. What does modality in data modeling indicates? Modality indicates whether or not a particular data object must participate in the relationship.

15. What is ERD?

Entity Relationship Diagram is the graphical representation of the object relationship pair. It is mainly used in database applications.

16. What is DFD?

Data Flow Diagram depicts the information flow and the transforms that are applied on the data as it moves from input to output.

17. What does Level0 DFD represent?

Level 0 DFD is called as „fundamental system model" or „context model". In the context model the entire software system is represented by a single bubble with input and output indicated by incoming and outgoing arrows.

18. What is a state transition diagram?

State transition diagram is basically a collection of states and events. The events cause the system to change its state. It also represents what actions are to be taken on the occurrence of particular event.

19. Define Data Dictionary.

The data dictionary can be defined as an organized collection of all the data elements of the system with precise and rigorous definitions so that user and system analyst will have a common understanding of inputs, outputs, components of stores and intermediate Calculations.

20. What are the elements of Analysis model?

i. Data Dictionary

ii. Entity Relationship Diagram

iii. Data Flow Diagram

iv. State Transition Diagram

v. Control Specification

vi. Process specification.

21. What are functional requirements?

Functional requirements are" statements of services the system should provide how the system should react to particular input and how the system should behave in particular situation.

22. What are non-functional requirements?

Non-functional requirements are constraints on the services or functions offered by the system such as timing constraints, constraints on the development process, standards, etc……..

23. What is the outcome of feasibility study?

The outcome of feasibility study is the results obtained from the following questions:

- Which system contributes to organizational objectives?
- Whether the system can be engineered? Is it within the budget?
- Whether the system can be integrated with other existing system?

24. What is meant by structural analysis?

The structural analysis is mapping of problem domain to flows and transformations. The system can be modeled by using Entity Relationship diagram, Data flow diagram and Control flow diagrams.

25. What is data dictionary? How is it used in software engineering?

A data dictionary is a collection of data about data. It maintains information about the definition, structure, and use of each data element that an organization uses. A data dictionary is invaluable for documentation purposes, for keeping control information on corporate data, for ensuring consistency of elements between organizational systems, and for use in developing databases.Data dictionary software packages are commercially available, often as part of a CASE package or DBMS. DD software allows for consistency checks and code generation. It is also used in DBMSs to generate reports.

26 Define Use case.

A use case is a collection of related success and failure scenarios that describe an actor using a system to support a goal. Use cases are text documents, not diagrams, and use-case modeling is

primarily an act of writing text, not drawing diagrams.

27. What are Three Kinds of Actors?

Primary actor, Supporting actor, offstage actor.

28. What Tests Can Help Find Useful Use Cases?

1. The Boss Test

2. The EBP Test

3. The Size Test 16.

29. What are Use Case Diagrams?

A use case diagram is an excellent picture of the system context; it makes a good context diagram that is, showing the boundary of a system, what lies outside of it, and how it gets used. It serves as a communication tool that summarizes the behavior of a system and its actors.

Part B (Possible Questions)

1. Write about the following requirements engineering activities.
   i. Inception
   ii. Elicitation
   iii. Elaboration
   iv. Negotiation

2. Explain the ways and means for collecting the software requirements and how are they organized and represented?

3. Explain in detail about Functional and nonfunctional requirements.

4. Explain in detail about user requirements? .

5. Explain in detail elicitation and analysis.

6. Explain about requirement validation and management.

7. Explain Petri nets and data dictionary.

8. What is the purpose of feasibility study.

   ii) State the inputs and results of the feasibility study.

   iii) List any four issues addressed by a feasibility study.

   iv) Elaborate the phases involved when carrying out a feasibility study .

9. Write about the relationship between sequence diagrams and use case?

Part C (Possible Questions)

1. Draw use case and Dataflow diagrams for a Restaurant System. The activities of the Restaurant system are listed below.

Receive the customer food orders, produce the customer ordered foods, serve the customer with their ordered foods ,collect payment from customers, store customer payment details, order raw materials for food products, pay for raw materials and pay for labor.

2. How do you prepare the Software documents for the following applications

   a. Bank management systems.

   b. Railway Ticket Reservation.

3. Draw the use case diagram for the following:A coffee machine dispense coffee to customers. Customers order coffee by selecting a recipe from a set of recipes. Customers pay for the bill using coins. Change is given back, if any.

   The service staff loads ingredients(coffee power,milk,sugar,water,chocolate) into the coffee machine. The service staff can also add a recipe by indicating the name of the coffee, the units coffee power, milk,sugar to be added as well as the cost of the coffee?

4. Describe the strategies used to identify the conceptual classes? Describe the steps to create a Domain model for representing the conceptual classes?